

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Interfacce di realtà aumentata per la Human-Robot Collaboration: sviluppo di un framework di streaming sensoriale basato su ROS 2 e validazione tramite algoritmi di SLAM

Relatore

Prof. Stefano Ghidoni

Laureando

Alessandro Rubert

Correlatore

Dott. Matteo Terreran, PhD

ANNO ACCADEMICO 2025-2026

Data di laurea 22/09/2026

“La semplicità è la massima raffinatezza.”

Leonardo da Vinci

Sommario

Negli ultimi anni, la Collaborazione Umano-Robot (Human-Robot Collaboration - HRC) ha assunto un ruolo centrale nell'industria 4.0 e nella robotica collaborativa, richiedendo canali di comunicazione sempre più bidirezionali, intuitivi ed efficienti. In questo contesto, la Realtà Aumentata (Augmented Reality, AR) si propone come una tecnologia abilitante fondamentale, capace sia di mostrare informazioni all'operatore sia di raccogliere dati spaziali sull'ambiente circostante. Questo lavoro di tesi presenta lo sviluppo di un framework di streaming sensoriale che mette in comunicazione il visore AR Magic Leap 2 con l'ecosistema ROS 2 (Robot Operating System). Sfruttando l'ambiente di sviluppo Unity e il pacchetto ROS-TCP-Connector, è stata realizzata un'applicazione a bordo del visore in grado di acquisire in tempo reale i dati provenienti dai sensori integrati (pose tracking, telecamere e sensori d'ambiente) e di pubblicarli su appositi topic ROS 2. Al fine di testare l'accuratezza, la latenza e la robustezza del flusso dati generato, il framework è stato validato attraverso l'utilizzo di algoritmi di SLAM (Simultaneous Localization and Mapping) noti in letteratura. I risultati sperimentali dimostrano che l'architettura proposta garantisce una trasmissione dati stabile ponendo le basi per un'interazione sicura in tempo reale e una consapevolezza spaziale condivisa e aprendo nuove prospettive per il controllo e la supervisione dei sistemi robotici in scenari di HRC complessi.

Indice

1	Introduzione	1
1.1	La robotica nelle imprese	1
1.1.1	Automazione rigida. L'industria 3.0	1
1.1.2	Automazione flessibile e collaborativa. L'industria 4.0	1
1.1.3	Human-Robot Collaboration (HRC)	3
1.2	Realtà aumentata e robotica: integrazione e limiti comunicativi	4
1.2.1	La realtà aumentata (AR)	4
1.2.2	AR come interfaccia con i robot	5
1.2.3	La mancanza di comunicazione nativa	5
1.3	Obiettivi	6
1.3.1	Il framework sviluppato	6
1.3.2	Validazione	6
1.4	Struttura della tesi	7
2	Stato dell'arte	9
2.1	La realtà aumentata nella collaborazione umano-robot (HRC)	9
2.2	Visione egocentrica e percezione per sistemi indossabili ed esoscheletri	10
2.3	Middleware di acquisizione e streaming sensoriale da visori AR	10
2.4	Il divario tecnologico e il ruolo di Magic Leap 2	11
3	Tecnologie utilizzate	13
3.1	Magic Leap 2	13
3.1.1	Configurazione hardware e dispositivi di input	14
3.1.2	Sensoristica	15
3.1.3	Sistema operativo	17
3.1.4	Magic Leap SDK	17
3.2	ROS 2 (Robot Operating System 2)	18
3.2.1	Architettura modulare	18

3.2.2	Sistemi di riferimento e trasformate	19
3.2.3	Strumenti	19
3.3	Unity	20
3.3.1	Architettura a componenti e ciclo di esecuzione	20
3.4	ROS-TCP-Connector e ROS-TCP-Endpoint	21
3.4.1	Architettura client-server	21
3.4.2	Serializzazione e generazione dei messaggi	21
3.5	ORB-SLAM3	22
3.5.1	Algoritmi di SLAM (Simultaneous Localization and Mapping)	22
3.5.2	ORB-SLAM3	23
4	Progettazione e implementazione del framework	25
4.1	Requisiti funzionali e vincoli	25
4.1.1	Requisiti funzionali	25
4.1.2	Vincoli di progetto e di sistema	26
4.2	Architettura del framework e negoziazione dei parametri	27
4.2.1	Componenti del sistema e flusso di negoziazione	27
4.2.2	Struttura del file di configurazione (config.yaml)	28
4.2.3	Architettura a componenti e modello di esecuzione in Unity	31
4.3	Implementazione e caratterizzazione dei moduli sensoriali	32
4.3.1	Telecamere di tracciamento ambientale (World Cameras)	33
4.3.2	Telecamera frontale RGB ad alta risoluzione (Picture Center)	35
4.3.3	Sensore di profondità (Depth camera)	38
4.3.4	Telecamere per l'Eye Tracking e stima dello sguardo	41
4.3.5	Unità inerziali (IMU) e sensore di luminosità ambientale	43
4.4	Manuale d'uso	45
4.4.1	Configurazione dell'ambiente ROS 2	45
4.4.2	Installazione e configurazione dell'applicazione su Magic Leap 2	46
4.4.3	Sequenza di avvio e streaming dei dati	46
4.5	Modularità e upgrade	47
4.5.1	Setup dell'ambiente di sviluppo Unity	47
4.5.2	Linee guida per l'estensione e l'upgrade del codice	48
5	Validazione dei risultati sperimentali	51
5.1	Setup sperimentale e procedura di acquisizione	51
5.2	Metodologia di analisi e segmentazione con evo	52
5.2.1	Motivazione della segmentazione dei dati	52

5.2.2	Metriche di errore e allineamento traiettorie	52
5.2.3	Decomposizione di traslazione e orientamento	53
5.3	Risultati sperimentali e analisi delle traiettorie	55
5.3.1	Validazione su World Center Camera — Traiettoria 1	55
5.3.2	Validazione su World Center Camera — Traiettoria 2	59
5.3.3	Validazione su World Left Camera — Traiettoria 3	63
5.3.4	Validazione su World Right Camera — Traiettoria 4	67
5.3.5	Considerazioni finali sui risultati	71
6	Conclusione e sviluppi futuri	73
6.1	Conclusioni	73
6.2	Sviluppi futuri	74
7	Ringraziamenti	77
	Bibliografia	79

Elenco delle figure

1.1	Cobot con umano	3
1.2	Esempi di Realtà Aumentata	4
3.1	Magic Leap 2	14
3.2	Lenti Magic Leap 2	15
3.3	Inquadratura frontale Magic Leap 2	16
3.4	Esempio di Transform.	19
4.1	Flusso di negoziazione dei parametri	28
4.2	Esempio di acquisizione dei fotogrammi in scala di grigi provenienti dalle tre telecamere di tracciamento ambientale (World Cameras): da sinistra verso destra, <i>World Left</i> , <i>World Center</i> e <i>World Right</i>	33
4.3	Flussi acquisiti dalla telecamera frontale RGB (<i>Picture Center</i>): a sinistra lo stream video convenzionale (<i>Stream 0</i>), a destra lo stream in realtà mista (<i>Stream 1</i>) con sovrapposizione degli elementi olografici dell'interfaccia utente.	35
4.4	Confronto visivo tra le quattro modalità operative del sensore di profondità Time-of-Flight (ToF): (a) Short Range con codifica metrica <i>Depth32</i> ; (b) Long Range con codifica metrica <i>Depth32</i> ; (c) Short Range con intensità infrarossa grezza <i>DepthRaw</i> ; (d) Long Range con intensità infrarossa grezza <i>DepthRaw</i>	38
4.5	Flussi a infrarossi acquisiti dalle quattro telecamere dedicate all'Eye Tracking: (a) <i>Eye Nasal Right</i> , (b) <i>Eye Nasal Left</i> , (c) <i>Eye Temple Right</i> e (d) <i>Eye Temple Left</i>	41
5.1	Ricostruzione 3D della traiettoria stimata e confronto con la ground truth, con mappa cromatica dell'errore APE di traslazione dopo allineamento Sim(3) (Umeyama).	55
5.2	Andamento temporale dell'Absolute Pose Error (APE) di traslazione con indicazione di RMSE, media, mediana e deviazione standard ($\pm\sigma$).	56
5.3	Proiezione bidimensionale sul piano <i>XY</i> della traiettoria stimata a confronto con la traiettoria di riferimento.	57

5.4	Evoluzione temporale delle singole componenti cartesiane di posizione ($x(t)$, $y(t)$, $z(t)$): confronto tra stima e ground truth.	57
5.5	Confronto temporale degli angoli di Eulero (Roll, Pitch, Yaw) stimati rispetto al riferimento nativo on-board del visore.	58
5.6	Ricostruzione 3D della traiettoria stimata e confronto con la ground truth, con mappa cromatica dell'errore APE di traslazione dopo allineamento Sim(3) (Umeyama).	59
5.7	Andamento temporale dell'Absolute Pose Error (APE) di traslazione con indicazione di RMSE, media, mediana e deviazione standard ($\pm\sigma$).	60
5.8	Proiezione bidimensionale sul piano XY della traiettoria stimata a confronto con la traiettoria di riferimento.	61
5.9	Evoluzione temporale delle singole componenti cartesiane di posizione ($x(t)$, $y(t)$, $z(t)$): confronto tra stima e ground truth.	61
5.10	Confronto temporale degli angoli di Eulero (Roll, Pitch, Yaw) stimati rispetto al riferimento nativo on-board del visore.	62
5.11	Ricostruzione 3D della traiettoria stimata e confronto con la ground truth, con mappa cromatica dell'errore APE di traslazione dopo allineamento Sim(3) (Umeyama).	63
5.12	Andamento temporale dell'Absolute Pose Error (APE) di traslazione con indicazione di RMSE, media, mediana e deviazione standard ($\pm\sigma$).	64
5.13	Proiezione bidimensionale sul piano XY della traiettoria stimata a confronto con la traiettoria di riferimento.	65
5.14	Evoluzione temporale delle singole componenti cartesiane di posizione ($x(t)$, $y(t)$, $z(t)$): confronto tra stima e ground truth.	65
5.15	Confronto temporale degli angoli di Eulero (Roll, Pitch, Yaw) stimati rispetto al riferimento nativo on-board del visore.	66
5.16	Ricostruzione 3D della traiettoria stimata e confronto con la ground truth, con mappa cromatica dell'errore APE di traslazione dopo allineamento Sim(3) (Umeyama).	67
5.17	Andamento temporale dell'Absolute Pose Error (APE) di traslazione con indicazione di RMSE, media, mediana e deviazione standard ($\pm\sigma$).	68
5.18	Proiezione bidimensionale sul piano XY della traiettoria stimata a confronto con la traiettoria di riferimento.	69
5.19	Evoluzione temporale delle singole componenti cartesiane di posizione ($x(t)$, $y(t)$, $z(t)$): confronto tra stima e ground truth.	69

5.20 Confronto temporale degli angoli di Eulero (Roll, Pitch, Yaw) stimati rispetto al riferimento nativo on-board del visore.	70
--	----

Capitolo 1

Introduzione

1.1 La robotica nelle imprese

1.1.1 Automazione rigida. L'industria 3.0

Tra gli anni '60 e gli anni '80 i primi robot industriali fanno la loro comparsa nelle fabbriche. Questi macchinari erano robusti, pesanti ed erano progettati per ripetere sistematicamente semplici operazioni e movimenti programmati senza percepire in alcun modo l'ambiente. Questo tipo di robotica si definisce “rigida” perché le linee sono dedicate ad un singolo prodotto e presentano elevati costi di riprogrammazione. Queste soluzioni tecnologiche erano utilizzate per sostituire l'umano nei compiti più gravosi, nocivi e ripetitivi come la manipolazione di pezzi incandescenti da pressofusione o la saldatura a punti. Successivamente, tra gli anni '80 e il 2000, i robot industriali raggiungono la maturità grazie alla diffusione dei microprocessori e dei primi sistemi di visione artificiale (computer vision). Ad esempio nascono in questo periodo i bracci articolati a 6 assi. Questi vengono successivamente dotati di telecamere che consentono di correggere i movimenti per adattarsi ai lievi disallineamenti dei pezzi. Grazie a questi cambiamenti i macchinari divengono più veloci e precisi. Tutte queste innovazioni hanno permesso di avere intere linee di produzione completamente automatizzate in grado di garantire precisione millimetrica e di lavorare anche di notte. Questi macchinari tuttavia, per ragioni di sicurezza, venivano rigorosamente segregati all'interno di gabbie metalliche o celle chiuse dotate di fotocellule e interlock che bloccavano la macchina in caso di apertura per evitare collisioni fatali con gli operatori e garantire la sicurezza funzionale.

1.1.2 Automazione flessibile e collaborativa. L'industria 4.0

A partire dai primi anni del 2000 le esigenze del mercato fanno diventare il paradigma dell'automazione rigida obsoleto. Infatti il mercato odierno è molto più dinamico. Un chiaro esempio

di questo mutamento è rappresentato dal settore automotive. Se storicamente la catena di montaggio era progettata per la produzione di un singolo modello con pochissime varianti (come le sole opzioni di colore), l'adozione di una linea di produzione rigida risultava la scelta ideale ed efficiente. Oggi invece le aziende automobilistiche lanciano sul mercato modelli di auto che arrivano ad avere migliaia di combinazioni di optional possibili. Altri esempi sono il mondo dell'E-commerce e del Fast Fashion. Ad esempio se un rossetto o una scarpa diventano virali su un social network la domanda passa da 100 a 50.000 pezzi in pochi giorni. Due settimane dopo, la moda è già finita e la linea configurata per incasellare i rossetti si ritrova a dover gestire forme completamente diverse. Anche nell'elettronica di consumo la produzione è estremamente dinamica. Le aziende che producono computer, smartphone o elettrodomestici lanciano un modello nuovo con forme e caratteristiche diverse ogni 9-12 mesi con conseguenti modifiche all'intera linea di produzione. In contesti così dinamici l'automazione rigida, che prevede fasi di progettazione di svariati mesi per poi smantellarla all'uscita del nuovo modello, risulta inapplicabile oltre che inefficiente e costosa. Pertanto è stato necessario un cambio di paradigma che è ancora oggi in atto. Occorre passare da un'automazione rigida ad un'automazione flessibile che riesca ad essere riprogrammata e riadattata alle esigenze in tempi brevissimi. Ciò è reso possibile dai robot collaborativi, ovvero macchinari interamente progettati per lavorare a stretto contatto con gli esseri umani senza arrecare loro danni. I cobot si distinguono per essere facilmente riprogrammabili dagli operatori umani, spesso anche tramite apprendimento intuitivo o interfacce semplificate. Qualora si presenti la necessità di rivoluzionare l'intera linea di produzione, queste macchine possono essere velocemente spostate, riconfigurate e adattate ai nuovi processi senza dover ricostruire da zero l'infrastruttura. Anche l'adozione di robot mobili che si spostano all'interno dell'azienda gestendo la logistica interna in modo dinamico ha aiutato questa transizione. Tra questi rientrano gli Automated Guided Vehicle (AGV) ovvero dei robot che seguono tracce fisse sul pavimento per muoversi e gli Autonomous Mobile Robot (AMR) che navigano in autonomia utilizzando algoritmi di localizzazione e mappatura (SLAM: Simultaneous Localization and Mapping). Questo consente di eliminare le gabbie metalliche e con lo spazio ricavato progettare layout di fabbrica più compatti e con capacità produttive più alte. Allo stesso tempo un operatore umano può lavorare a stretto contatto con il robot. L'umano può svolgere operazioni che per il robot sarebbero molto difficili e il robot si può occupare di attività che per l'umano sarebbero estremamente faticose. Ad esempio nella produzione di un dispositivo elettronico il robot collaborativo può avvitare le micro-viti che per l'umano sarebbe noioso e logorante, nel frattempo l'umano controlla che i chip siano posizionati correttamente e che i cavi flessibili seguano i percorsi corretti (operazioni che sarebbero molto difficili per un robot). Certamente, per la produzione su larga scala di alcuni prodotti l'automazione rigida resta la soluzione migliore ma nelle industrie più dinamiche poter spostare e riprogrammare i robot

in modo rapido è molto più efficace.



Figura 1.1: Esempio di robot collaborativo (cobot) integrato in una postazione di lavoro a condivisione di spazio con l'operatore umano. Tratta da [1].

1.1.3 Human-Robot Collaboration (HRC)

Alla base dell'industria 4.0 c'è la Human-Robot collaboration che si basa sulla combinazione delle migliori qualità di entrambi i soggetti. Gli umani apportano flessibilità, intuito e capacità decisionale di fronte agli imprevisti, mentre i cobot garantiscono precisione millimetrica, forza e costanza nelle mansioni ripetitive e gravose. Dalla loro unione nasce un processo produttivo efficiente, sicuro e ad alto valore aggiunto. Per garantire un'interazione che sia al contempo sicura ed efficiente, emerge la stretta necessità di implementare un'interfaccia di comunicazione bidirezionale. Il nodo critico di tale integrazione risiede infatti nella reciproca trasparenza intenzionale: da un lato, il robot deve prevedere e comprendere le azioni e le intenzioni dell'umano (tramite modelli di intent recognition e tracciamento dei movimenti) per adattare la propria traiettoria in modo proattivo; dall'altro, l'operatore deve poter anticipare chiaramente i movimenti futuri del robot per evitare collisioni, diminuire il carico cognitivo e instaurare un rapporto di fiducia. In questo scenario, la realtà aumentata (AR) si configura come una tra le interfacce più promettenti.

1.2 Realtà aumentata e robotica: integrazione e limiti comunicativi

1.2.1 La realtà aumentata (AR)

La Realtà Aumentata (Augmented Reality, AR) è una tecnologia che potenzia la percezione del mondo esterno. Tramite dispositivi dedicati (come visori o smartphone) vengono riprodotti elementi digitali come immagini 3D e ologrammi che si sovrappongono all'ambiente fisico circostante. A differenza della Realtà Virtuale (Virtual Reality, VR), che sostituisce interamente l'ambiente reale con una simulazione sintetica che isola l'utente, la Realtà Aumentata ne integra e arricchisce il campo visivo o sensoriale con diversi tipi di elementi sensoriali (grafici, uditivi o aptici). Un esempio pratico di applicazione quotidiana è offerto dal sito web di IKEA[2], che consente di visualizzare i propri prodotti direttamente all'interno della propria abitazione attraverso la modalità AR sul proprio smartphone (Figura 1.2).



(a) Immagine reale senza AR



(b) AR prima angolazione



(c) AR seconda angolazione

Figura 1.2: Sovrapposizione di elementi 3D via AR tramite il sito IKEA (adattato da [2], screenshot dell'autore).

1.2.2 AR come interfaccia con i robot

L'integrazione della Realtà Aumentata (AR) nella robotica collaborativa ridefinisce l'interazione umano-macchina, trasformando un visore in un vero e proprio ponte bidirezionale e sistema sensoriale avanzato. Tradizionalmente, la percezione spaziale nelle postazioni di lavoro con robotica collaborativa è affidata a complessi e costosi sistemi d'ambiente costituiti da numerose telecamere fisse installate lungo il perimetro di lavoro che consentono al robot di sapere dove si trova e cosa sta facendo l'operatore. Al contrario, un visore AR indossato dall'operatore semplifica drasticamente questa architettura: le telecamere e i sensori di profondità integrati nel dispositivo tracciano continuamente la posizione della testa, le mani e l'ambiente circostante in tempo reale. In questo modo, è il visore stesso a "vedere" lo spazio per conto del robot, ricostruendo la mappa 3D del contesto dinamico direttamente dalla prospettiva dell'utente. Inoltre il robot non si limita a ricevere solo comandi o parametri di traiettoria, ma invia a sua volta dati sullo stato di avanzamento e intenzioni di movimento che vengono proiettati direttamente nel campo visivo dell'operatore, garantendo un flusso informativo continuo, condiviso e privo di infrastrutture esterne ingombranti.

1.2.3 La mancanza di comunicazione nativa

Sebbene i visori AR di fascia alta presenti sul mercato, come il Magic Leap 2 che è stato utilizzato in questo lavoro di tesi, offrano avanzate capacità di tracciamento spaziale e percezione dell'ambiente, questi dispositivi non dispongono di un'integrazione nativa con gli ecosistemi robotici standard come ROS 2 (Robot Operating System). Per colmare questo divario architetturale si rende necessaria l'implementazione di un middleware di comunicazione intermedio. Questo "ponte" software ha il compito di estrarre in tempo reale dai dispositivi AR i dati raccolti dal visore, principalmente i flussi video, per poi convertirli e trasmetterli al sistema robotico. In questo modo, il robot può utilizzare questi dati per ottimizzare la pianificazione delle traiettorie, migliorare la localizzazione o eseguire compiti complessi di manipolazione e collaborazione umano-robot.

1.3 Obiettivi

1.3.1 Il framework sviluppato

L'obiettivo principale di questo lavoro di tesi, svolto presso lo IAS-Lab dell'Università di Padova, è la progettazione e lo sviluppo di un framework software in grado di colmare il divario di comunicazione tra il visore di Realtà Aumentata di fascia alta Magic Leap 2 e l'ecosistema robotico ROS 2 (Robot Operating System). Per superare la mancanza di un'integrazione nativa, è stata realizzata un'architettura modulare fondata sulla cooperazione in rete tra il dispositivo indossabile e un computer (che ipoteticamente si occupa di gestire altre apparecchiature come bracci o telecamere) con installato ROS 2. La comunicazione tra i due ambienti può avvenire sia in modalità wireless sia tramite connessione cablata, garantendo in quest'ultimo caso la massima stabilità di banda e valori di latenza estremamente ridotti per le applicazioni più critiche. Il funzionamento del framework si basa su una gestione dinamica e ottimizzata delle risorse sensoriali:

- Configurazione dinamica da ROS 2: Sul computer risiede un nodo ROS 2 dedicato che legge un file di configurazione in formato `.yaml`. All'interno di questo file vengono definiti i parametri operativi della comunicazione, quali la selezione dei sensori da abilitare (es. telecamere, dati inerziali, sensori di profondità) e la risoluzione dei flussi dati necessari.
- Avvio dello streaming su Magic Leap 2: All'avvio dell'applicazione sul visore, quest'ultimo riceve la configurazione inviata dal computer e adatta la propria elaborazione di conseguenza, avviando lo streaming dei soli dati richiesti. Questo approccio evita il saturamento della banda di rete ed evita inutili carichi computazionali sul visore per sensori non utilizzati.

Il lavoro svolto ha portato alla realizzazione di un'applicazione completa e pronta all'uso, concepita fin dall'inizio con un'architettura altamente scalabile e predisposta per essere facilmente estesa con nuove funzionalità o ulteriori moduli di elaborazione dati. Infine, nell'ottica di favorire la riproducibilità scientifica e incentivare la collaborazione open-source, l'intero progetto è stato rilasciato in un repository pubblico su GitHub [3]. In questo modo, la comunità scientifica e gli sviluppatori potranno replicare l'infrastruttura proposta, testarla con altri sistemi robotici o arricchirla integrando nuove feature.

1.3.2 Validazione

Al fine di valutare la correttezza operativa del framework sviluppato e la qualità dei flussi dati trasmessi in tempo reale, è stata condotta una fase di validazione sperimentale. In particolare,

si è voluto verificare l'effettiva usabilità dei dati sensoriali all'interno di un tipico algoritmo di localizzazione e mappatura robotica. A tal scopo, i flussi video generati dalle world camera del Magic Leap 2 e pubblicati su ROS 2 tramite il framework sono stati forniti in ingresso ad ORB-SLAM3 [4], uno dei sistemi di SLAM (Simultaneous Localization and Mapping) di riferimento in letteratura, utilizzato per l'occasione in modalità monoculare. Per valutare le prestazioni dell'algoritmo, la traiettoria stimata da ORB-SLAM3 è stata posta a confronto con la ground truth offerta dalle pose ed estrinseche stimate internamente dal visore, scaturite dal tracciamento spaziale nativo del Magic Leap 2, noto per la sua elevata accuratezza e precisione. La sperimentazione è stata condotta eseguendo diversi test su traiettorie a breve raggio in ambiente indoor. I risultati del confronto tra la traiettoria ricostruita in modalità monoculare e la stima nativa del dispositivo hanno confermato il corretto instradamento dei dati, la ridotta latenza di trasmissione e la piena compatibilità dell'infrastruttura sviluppata con gli algoritmi di stima dello stato comunemente impiegati in ambito robotico.

1.4 Struttura della tesi

Il presente lavoro di tesi si articola nei capitoli successivi in modo progressiva. Nel Capitolo 2 viene trattato lo stato dell'arte, analizzando il ruolo della Realtà Aumentata nella cooperazione umano-robot, le soluzioni di visione egocentrica per sistemi indossabili ed esoscheletri, e i principali middleware di streaming sensoriale, contestualizzando il divario tecnologico che il visore Magic Leap 2 si propone di colmare. Il Capitolo 3 introduce le tecnologie impiegate nel progetto, descrivendo l'architettura hardware e la sensoristica del Magic Leap 2, i fondamenti di ROS 2, l'ambiente Unity, il connettore ROS-TCP-Connector e il funzionamento dell'algoritmo di localizzazione ORB-SLAM3. Nel Capitolo 4 viene approfondita la progettazione e l'implementazione del framework, illustrando la definizione dei requisiti, il meccanismo di negoziazione dinamica dei parametri tramite file di configurazione, la caratterizzazione dei singoli flussi sensoriali e le linee guida per l'utilizzo e l'estendibilità del codice. Il Capitolo 5 espone la validazione sperimentale del middleware, illustrando la metodologia di valutazione delle traiettorie tramite il package evo e confrontando i risultati dello SLAM monoculare con la stima di posa nativa del visore lungo diversi percorsi di prova. Infine, il Capitolo 6 riassume i principali traguardi raggiunti e trae le conclusioni del lavoro, dando una visione sulle prospettive di sviluppo futuro per l'interazione bidirezionale e la robotica collaborativa avanzata.

Capitolo 2

Stato dell'arte

2.1 La realtà aumentata nella collaborazione umano-robot (HRC)

Negli ultimi anni, la transizione verso il paradigma dell'Industria 4.0 ha reso la cooperazione diretta tra umano e macchina un requisito cardine per aumentare la flessibilità e l'efficienza delle linee produttive. In questo scenario, la sicurezza dell'operatore e la fluidità dell'interazione rappresentano le due sfide principali. Tradizionalmente, la salvaguardia dell'umano nei contesti di Human-Robot Collaboration (HRC) viene gestita tramite recinzioni virtuali, sensori perimetrali o meccanismi di arresto controllato conformi agli standard ISO/TS 15066. Tuttavia, l'arresto passivo del robot in presenza dell'operatore penalizza fortemente la produttività complessiva. La Realtà Aumentata (AR) si è affermata come la tecnologia più promettente per superare questa dicotomia, fornendo canali di comunicazione intuitivi e bidirezionali. L'impiego di interfacce AR consente di visualizzare in tempo reale nel campo visivo dell'operatore lo stato interno del robot, le traiettorie pianificate, i volumi di sicurezza dinamici e le istruzioni procedurali passo-passo. Uno studio rilevante in ambito industriale è quello condotto da Hietanen et al. [5] (2020), in cui è stata sviluppata un'interfaccia interattiva basata su AR (confrontando sistemi a proiezione e visori indossabili come HoloLens) per la divisione dinamica dello spazio di lavoro durante l'assemblaggio di componenti di motori diesel. La ricerca ha dimostrato che la condivisione visiva delle zone di sicurezza e delle intenzioni del manipolatore riduce i tempi morti del robot di oltre il 50% e i tempi complessivi di esecuzione del 20-24% rispetto alle postazioni non collaborative. Parallelamente, l'AR si è evoluta da semplice strumento di visualizzazione passiva a mezzo attivo di controllo e insegnamento (Learning from Demonstration, LfD). In questo contesto, Yan et al. [6] (2024) hanno proposto un framework complementare per HRC che combina un visore AR (HoloLens 2) e un'interfaccia aptica per il controllo di un manipolatore a 7 gradi di libertà (Franka Emika). L'operatore può intervenire nello spazio nullo

del robot tramite gesture aeree tracciate dal visore, modificando la postura del braccio per evitare collisioni impreviste senza interrompere il compito primario dell'end-effector, e insegnando nuove traiettorie parametriche modellate tramite Dynamic Movement Primitives (DMP).

2.2 Visione egocentrica e percezione per sistemi indossabili ed esoscheletri

Oltre ai robot industriali fissi, i dispositivi di Realtà Aumentata trovano crescente applicazione nei sistemi robotici mobili e indossabili, in particolare negli esoscheletri per arti inferiori (Lower-Limb Exoskeletons, LLE). Gli esoscheletri assistivi tradizionali basano il proprio funzionamento su pattern di camminata predefiniti derivati dalla biomeccanica standard, risultando rigidi e poco adatti ad ambienti non strutturati. Per garantire un'adattabilità reale al terreno, è indispensabile dotare questi sistemi di capacità di percezione ambientale autonoma (Environment-Adaptive Gait Planning, EAGP). In quest'ottica, Trombin et al. [7] (2024) hanno introdotto un generatore di traiettorie privo di collisioni (CFFTG) in grado di adattare l'altezza e la lunghezza del passo in tempo reale basandosi sull'analisi di nuvole di punti 3D acquisite da telecamere RGB-D. Tuttavia, i soli sensori montati a livello del bacino o dei piedi presentano limitazioni intrinseche: hanno un campo visivo limitato verso il basso e rilevano gli ostacoli solo a brevissima distanza, impedendo una pianificazione anticipata del cammino. Per superare tale vincolo, Mihailovic et al. [8] (2025) hanno proposto un modulo di visione egocentrica che integra una telecamera montata su visore AR (head-mounted) con la camera dell'esoscheletro. Sfruttando algoritmi di SLAM visivo-inerziale (ORB-SLAM3) eseguiti in parallelo sui flussi e tecniche di corrispondenza geometrica 3D (DUST3R), il sistema unifica le mappe generate dalle due prospettive. Questa configurazione estende l'orizzonte percettivo dell'utente, consente il riconoscimento precoce di scale, rampe e ostacoli, e al contempo proietta ologrammi di feedback (come i punti di appoggio sicuri per i piedi) direttamente sul display ottico del visore.

2.3 Middleware di acquisizione e streaming sensoriale da visori AR

Affinché un visore AR possa operare efficacemente come nodo sensoriale all'interno di architetture di controllo robotico complesse, è necessario estrarre i dati grezzi dei sensori di bordo a frequenze elevate e con latenze minime verso workstation esterne. Nei primi anni di diffusione dei visori commerciali, l'accesso all'hardware di basso livello era fortemente limitato dai sistemi operativi proprietari. Una svolta fondamentale per la comunità scientifica è avvenuta con

l'introduzione del "Research Mode" per Microsoft HoloLens 2, documentato da Ungureanu et al. [9] (2020). Il Research Mode ha fornito per la prima volta un set di API a basso livello in C++ per accedere direttamente ai flussi delle quattro telecamere monocromatiche a infrarossi per il tracking (VLC), al sensore di profondità a tempo di volo (ToF) nelle modalità a corto e lungo raggio, e ai campioni grezzi dell'unità inerziale (IMU). Sfruttando tali API, Dibene e Dunn [10] (2022) hanno sviluppato hl2ss (HoloLens 2 Sensor Streaming), un'applicazione server basata su socket TCP per Universal Windows Platform (UWP) in grado di trasmettere in tempo reale verso client Python tutti i flussi visivi, le pose spaziali estrapolate dal tracciamento della testa e i dati di eye e hand tracking. Il lavoro ha dimostrato l'efficacia dello streaming su rete locale per eseguire compiti computazionalmente onerosi (come la segmentazione panottica o la ricostruzione 3D densa TSDF) su workstation dedicate.

2.4 Il divario tecnologico e il ruolo di Magic Leap 2

Nonostante il successo e l'ampia adozione di Microsoft HoloLens 2 nella letteratura scientifica dell'ultimo quadriennio, il panorama tecnologico ha subito una profonda trasformazione: il dispositivo HoloLens 2 è giunto al termine del proprio ciclo di vita commerciale ed è uscito di produzione, lasciando la comunità di ricerca priva di una piattaforma di riferimento supportata a lungo termine basata su Windows UWP. Il visore Magic Leap 2 rappresenta il naturale successore tecnologico di riferimento per la realtà aumentata di livello professionale e industriale:

- **Hardware di Percezione Avanzato:** Integra tre telecamere di tracciamento monocromatiche ad ampio FOV e ad alta risoluzione (1016×1016 a 30 Hz), una telecamera RGB centrale fino a 4K, un sensore di profondità ToF metrico, quattro telecamere IR per l'Eye Tracking e tre unità inerziali distinte.
- **Architettura Software Aperta:** Basato su sistema operativo Magic Leap OS (derivato da Android AOSP) e compatibile con lo standard industriale OpenXR.

Tuttavia, a differenza del panorama HoloLens, nell'ecosistema Magic Leap 2 non esisteva uno strumento open-source dedicato allo streaming centralizzato, modulare e sincronizzato dei sensori grezzi verso il middleware robotico standard ROS 2. Il presente lavoro di tesi si inserisce esattamente per colmare questo divario tecnologico: progettando un bridge modulare basato su Unity e plugin Android nativi, in grado di negoziare dinamicamente i parametri di trasmissione da ROS 2 e validare l'integrità temporale e spaziale dei flussi video tramite pipeline di localizzazione visiva in tempo reale.

Capitolo 3

Tecnologie utilizzate

In questo lavoro di tesi è stato sviluppato un framework per consentire lo streaming in tempo reale dei dati raccolti dal visore AR Magic Leap 2 verso un computer equipaggiato con ROS 2. Il sistema è stato realizzato come un'applicazione in ambiente Unity per gestire l'interfaccia grafica e l'acquisizione dei flussi sensoriali a bordo del visore. La comunicazione tra Unity e ROS 2 è affidata al pacchetto ROS-TCP-Connector, mentre la qualità e la sincronizzazione dei dati trasmessi sono state verificate tramite l'algoritmo di localizzazione ORB-SLAM3. In questo capitolo vengono descritti i principali strumenti tecnologici impiegati, evidenziando le loro relazioni e il ruolo che ciascuno riveste all'interno del progetto.

3.1 Magic Leap 2

In questo lavoro di tesi è stato utilizzato il Magic Leap 2, un visore avanzato per la realtà aumentata, progettato dall'azienda americana Magic Leap e indirizzato prevalentemente al mercato aziendale e professionale (come chirurgia, ingegneria, manifattura e difesa). Nel panorama dei visori occorre differenziare il Magic Leap 2 dai visori più famosi inquadrando 3 tipi di approcci alla realtà aumentata:

- Visori VR con Passthrough Video (Apple Vision Pro, Meta Quest, HTC Vive XR Elite, Pimax Crystal)
- Smart Glasses tradizionali (Ray-Ban Meta Display)
- Veri visori AR con *Optical See-Through* (Magic Leap 2)

La prima categoria di visori comprende dispositivi che nascono per la realtà virtuale. L'utente quando indossa il dispositivo osserva degli schermi e la visione del mondo esterno viene ricostruita artificialmente riprendendo l'ambiente tramite telecamere. Ciò introduce inevitabilmente

una latenza di elaborazione (delay) e una risoluzione visiva inferiore rispetto alla vista umana diretta. Questa discrepanza sensoriale è la causa primaria di affaticamento visivo e *motion sickness* (chinetosi da realtà virtuale), limitandone l'uso prolungato. La seconda categoria di visori è priva di sensori avanzati per la mappatura tridimensionale dell'ambiente (SLAM), questi dispositivi non sono in grado di effettuare un tracciamento spaziale. Di conseguenza, gli elementi grafici rimangono sovrapposti al campo visivo e seguono i movimenti della testa, rendendo impossibile la visualizzazione di oggetti virtuali ancorati e fermi nello spazio. Il Magic Leap 2 (Figura 3.1) consente all'utente di osservare l'ambiente circostante in modo diretto attraverso lenti trasparenti (simili a occhiali da sole) e proietta nell'ambiente oggetti 3D localizzati nello spazio: muovendosi nella stanza, gli elementi virtuali rimangono perfettamente ancorati e fissi nella loro posizione.



Figura 3.1: Visore AR Magic Leap 2. Tratta da [11].

3.1.1 Configurazione hardware e dispositivi di input

L'hardware del Magic Leap 2 adotta un'architettura distribuita: la montatura degli occhiali non ospita direttamente l'unità di elaborazione principale né il pacchetto di alimentazione. L'esecuzione dei complessi algoritmi di Simultaneous Localization and Mapping (SLAM), l'elaborazione della grafica computazionale e la gestione delle applicazioni sono interamente delegate ad un "compute module" esterno collegato via cavo, il quale racchiude l'unità di calcolo ad alte prestazioni e la batteria (Figura 3.1). Per garantire un'interazione precisa, fluida ed ergonomica all'interno dell'ambiente di realtà aumentata, il sistema include inoltre un controller dedicato dotato di tracciamento avanzato, concepito per ottimizzare l'esperienza d'uso e la manipolazione degli elementi virtuali (Figura 3.1). Il fiore all'occhiello dell'azienda Magic Leap sono le sue lenti a guide d'onda diffrattive (*Waveguides*). Queste lenti hanno al loro interno svariati strati di

vetro ultra-sottile su cui sono incise micro-strutture tramite nanolitografia. Un micro-proiettore (motori ottici LCOS) posizionato sulla montatura proietta immagini digitali verso il bordo della lente. La luce proiettata entra nella guida d'onda e viaggia al suo interno fino a raggiungere delle reticolazioni diffrattive stampate sulle lenti che piegano la luce verso l'occhio dell'utente, sovrapponendo gli ologrammi alla vista del mondo reale [12]. Inoltre, le lenti di questo visore hanno uno strato dedicato al *Dynamic Dimming*. Si tratta di un layer a cristalli liquidi in grado di controllare il passaggio della luce che può oscurare l'intero ambiente o creare una “maschera d'ombra” nera esattamente dietro agli ologrammi. Il risultato è che gli ologrammi non appaiono semi trasparenti anche quando ci si trova in stanze molto illuminate o sotto la luce diretta del sole.



Figura 3.2: Lenti a guida d'onda diffrattiva del Magic Leap 2. Tratta da [13].

3.1.2 Sensoristica

Il Magic Leap 2 presenta svariati sensori che combinati consentono al visore di localizzarsi nello spazio con una precisione millimetrica e di conseguenza riesce a calcolare in tempo reale dove proiettare gli oggetti sulle lenti affinché appaiano ancorati alla realtà. In particolare, l'architettura sensoriale del dispositivo comprende:

- Unità di Misura Inerziale (IMU): sensori accelerometrici e giroscopici per tracciare la rotazione e l'accelerazione dei movimenti della testa ad altissima frequenza. In particolare ci sono due IMU all'interno dell'headset e uno nel compute module.
- Telecamere di tracciamento (World Camera): tre camere in scala di grigi di risoluzione 1016x1016 posizionate una al centro, una a destra e una a sinistra dell'headset (In rosso in figura 3.3).

- Telecamera RGB (Picture): una camera ad alta risoluzione posizionata frontalmente al centro del visore (In verde in figura 3.3).
- Sensore di profondità (Depth Camera): camera ToF (Time of Flight) per ricostruire con precisione la geometria 3D delle superfici e la distanza degli oggetti attorno all'utente. Posizionata frontalmente e al centro dell'headset con risoluzione 544x480 (In viola in figura 3.3).
- Telecamere di Eye Tracking: fotocamere dedicate agli occhi per monitorare lo sguardo dell'utente, ottimizzare il rendering e calcolare con esattezza la convergenza visiva. Risoluzione 400x400 posizionate 2 per occhio una vicino al naso e una vicino alla tempia (In azzurro in figura 3.3).
- Sensori ambientali e di illuminazione: per calcolare le condizioni di luce esterne e regolare la luminosità e l'oscuramento (dimming) delle lenti.
- Sensori di pressione: per calcolare con maggiore precisione gli spostamenti verticali del visore.

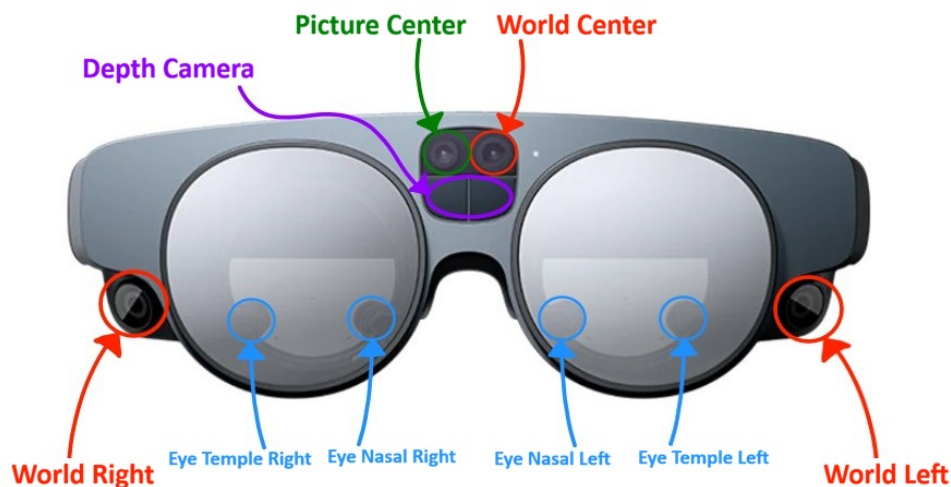


Figura 3.3: Inquadratura frontale del Magic Leap 2.

3.1.3 Sistema operativo

Il Magic Leap 2 è equipaggiato con Magic Leap OS, un sistema operativo proprietario basato su Android (*AOSP - Android Open Source Project*). La scelta di adottare un'architettura derivata da Android consente di combinare la stabilità di un kernel Linux ampiamente testato con un ecosistema di sviluppo nativo in C/C++ e Java, facilitando al contempo l'integrazione con engine grafici come Unity. A differenza dei sistemi operativi mobile convenzionali, Magic Leap OS è stato fortemente personalizzato a basso livello per gestire la complessa architettura distribuita del visore. Il sistema operativo gestisce in tempo reale il bilanciamento del carico computazionale tra le unità di elaborazione, la gestione della calibrazione ottica (incluso il controllo dinamico del dimming) e l'esecuzione degli algoritmi di localizzazione, indispensabili per la realtà aumentata.

3.1.4 Magic Leap SDK

L'SDK (Software Development Kit) per Magic Leap 2 è il set di strumenti, API e librerie fornito da Magic Leap per consentire agli sviluppatori di creare applicazioni di Realtà Aumentata (AR) vicine agli standard industriali come OpenXR (a differenza del suo predecessore Magic Leap). Il developer kit fornisce 3 strade per sviluppare applicazioni su Magic Leap 2:

- Unity (Engine raccomandato): È la piattaforma più supportata. Il pacchetto Magic Leap Unity SDK si integra tramite il sistema standard OpenXR ed XR Plug-in Management di Unity.
- Unreal Engine 5 (UE5): È presente un Magic Leap Unreal SDK per Unreal Engine 5 che sfrutta le estensioni OpenXR.
- C/C++ Native SDK: Per chi sviluppa engine proprietari o ha bisogno di prestazioni/integrazioni a bassissimo livello.

Le funzionalità principali che l'SDK fornisce sono:

- Mappatura Spaziale (Spatial Mapping): Scansiona l'ambiente 3D riconoscendo pareti, pavimenti e mobili per gestire collisioni fisiche, rilevare superfici piane e posizionare oggetti virtuali in modo permanente (Spatial Anchors).
- Tracciamento Corporeo (Eye e Hand Tracking): Monitora sguardo, pupille e movimenti avanzati delle mani (articolazioni e gesti) per consentire l'interazione diretta senza dispositivi esterni.
- Controller 6 DoF: Offre un tracciamento preciso a 6 gradi di libertà integrando sensori ottici ed elettromagnetici per eliminare i punti ciechi.

- **Audio Spaziale:** Simula la provenienza e la propagazione del suono in 3D per un'esperienza acustica immersiva e coerente con lo spazio circostante.
- **Accesso ai Sensori:** Fornisce agli sviluppatori l'accesso ai dati raccolti dall'hardware (telecamere, sensori di profondità, IMU, microfoni e luce ambientale) per analisi avanzate o algoritmi custom.

Magic Leap rende disponibile la documentazione ufficiale per l'utilizzo dell'SDK [14].

3.2 ROS 2 (Robot Operating System 2)

ROS 2 (Robot Operating System 2) è il framework middleware open source standard de facto per la progettazione e lo sviluppo di software robotico professionale e industriale. Nonostante il nome, non è un sistema operativo tradizionale, ma un set di librerie, strumenti e convenzioni che permettono a componenti software distinti di comunicare tra loro, gestendo sia i singoli sensori/attuatori sia intere flotte di robot.

3.2.1 Architettura modulare

ROS 2 si basa su un'architettura software modulare. Lo standard su cui si basa la comunicazione su ROS 2 è DDS (*Data Distribution Service*) ovvero uno standard industriale aperto di tipo data-centric sviluppato dall'Object Management Group (OMG) per la comunicazione peer-to-peer in tempo reale. A differenza del predecessore ROS 1, che faceva affidamento su un nodo di coordinamento centrale (ROS Master) e su protocolli interni personalizzati, DDS garantisce un'architettura completamente distribuita e decentralizzata. L'unità computazionale fondamentale è il nodo, un processo indipendente deputato a un singolo compito (es. lettura sensori, controllo attuatori). I nodi possono scoprirsi reciprocamente in modo automatico sulla rete e scambiare dati con un elevato livello di affidabilità e determinismo, permettendo inoltre di definire criteri avanzati di Qualità del Servizio (*Quality of Service - QoS*) per regolare priorità, tolleranza ai guasti, persistenza e gestione della banda nei flussi di comunicazione. Per scambiarsi dati, i nodi utilizzano due principali paradigmi di comunicazione: i topic, canali asincroni di tipo Publish-Subscribe (molti-a-molti) ideali per flussi continui di informazioni come gli stream sensoriali. Il framework ha altri due tipi di protocolli di comunicazione ovvero i servizi e le azioni. I servizi funzionano come una richiesta istantanea. Un nodo chiede un'operazione e aspetta subito una risposta prima di andare avanti. È il sistema perfetto per compiti brevi e immediati, come accendere un LED, resettare un sensore o chiedere la carica attuale della batteria. Le azioni servono per compiti lunghi e complessi come potrebbe essere guidare un robot da un punto ad un altro. A differenza dei servizi, durante l'esecuzione l'azione invia dei feedback in

tempo reale sullo stato dell'operazione e permette di annullare il compito in qualsiasi momento prima di restituire il risultato finale.

3.2.2 Sistemi di riferimento e trasformati

Per tracciare posizioni e movimenti del robot nel tempo, ROS 2 utilizza l'infrastruttura TF (Transform). Una TF rappresenta la posizione e l'orientamento di un elemento rispetto a un altro (Figura 3.4), corrispondente matematicamente a una trasformazione affine rigida in $SE(3)$. Essa definisce la relazione tra due sistemi di riferimento (frames) tramite:

- un vettore tridimensionale $\mathbf{t} = [x, y, z]^T$ per la traslazione;
- un quaternion unitario $\mathbf{q} = [q_x, q_y, q_z, q_w]^T$ per la rotazione 3D.

Le trasformazioni sono organizzate in una TF tree, una struttura ad albero i cui nodi sono i sistemi di riferimento(frames) e gli archi le relative trasformazioni. Ciò garantisce un unico percorso tra due frames qualsiasi, consentendo a ROS 2 di calcolare trasformazioni composte e convertire coordinate 3D all'istante.



Figura 3.4: Rappresentazione delle trasformazioni (TF) delle camere del visore Magic Leap 2 rispetto al sistema di riferimento del Magic Leap (world frame) e rispetto a un marcatore visivo denominato tag4. Screenshot dei dati visualizzati tramite RViz.

3.2.3 Strumenti

ROS 2 mette a disposizione diversi strumenti essenziali per lo sviluppo e il debugging, tra cui:

- RViz 2: uno strumento di visualizzazione 3D che consente di monitorare in tempo reale i dati pubblicati sui topic (ad esempio TF, point cloud, immagini).
- rosbag2: un tool di registrazione e riproduzione (replay) dei flussi di dati scambiati tra i nodi, utile per analizzare i test anche in assenza dell'hardware reale.

3.3 Unity

Unity è un motore grafico 3D (game engine) e ambiente di sviluppo multiplatforma in tempo reale, ampiamente impiegato per la realizzazione di contenuti interattivi e applicazioni di Realtà Aumentata (AR) e Virtuale (VR). All'interno di questo lavoro di tesi, è stata utilizzata la versione Unity 2022.3.62f3 LTS (Long Term Support). Unity non è stato impiegato come simulatore, ma come ambiente di runtime ed esecuzione principale a bordo del Magic Leap 2, deputato al rendering grafico degli elementi olografici e alla gestione dell'applicazione che acquisisce i dati sensoriali del visore.

3.3.1 Architettura a componenti e ciclo di esecuzione

Unity si struttura su un paradigma a componenti basato su una gerarchia di oggetti:

- **GameObject e Componenti:** Qualsiasi elemento presente nell'ambiente virtuale è un `GameObject`. Il comportamento e le caratteristiche dell'oggetto vengono definiti associandovi componenti modulari (come fotocamere virtuali, trasformazioni spaziali `Transform` o `collider`). Le logiche applicative personalizzate vengono implementate tramite script in linguaggio C# che estendono la classe base `MonoBehaviour`.
- **Main Thread e Ottimizzazione del Frame Rate:** Il runtime di Unity elabora la logica di gioco, l'interazione con le API del dispositivo e il rendering sul Main Thread. Nelle applicazioni di realtà aumentata ottica, mantenere un frame rate elevato e costante (solitamente tra i 60 e i 120 fps) è critico per evitare discrepanze visive tra il mondo reale e gli ologrammi sovrapposti; di conseguenza, l'elaborazione dei buffer sensoriali deve essere gestita in modo non bloccante.

L'esecuzione del codice segue un ciclo di vita a eventi deterministico scandito dal rendering del motore grafico. Tra i metodi principali utilizzati per l'interfacciamento e la gestione dei flussi dati figurano:

- **`Awake()` e `Start()`:** Metodi di inizializzazione invocati alla creazione dell'oggetto, impiegati per allocare le risorse in memoria e avviare i servizi di cattura dei sensori.
- **`Update()`:** Invocato a ogni ciclo di rendering (una volta ogni frame) a frequenza variabile in base al carico computazionale; è deputato all'aggiornamento dello stato dei componenti e alla gestione degli eventi in tempo reale.
- **`FixedUpdate()`:** Eseguito a intervalli temporali regolari e discreti (Δt fisso), indipendente dal frame rate di rendering, impiegato per la fisica e per operazioni che richiedono una periodicità temporale costante.

3.4 ROS-TCP-Connector e ROS-TCP-Endpoint

Per consentire lo scambio bidirezionale di messaggi tra l'applicazione Unity in esecuzione a bordo del Magic Leap 2 e i nodi ROS 2 attivi sulla workstation di calcolo, è stato impiegato il framework open source ROS-TCP-Connector [15], sviluppato all'interno del progetto Unity Robotics Hub. Questo strumento funge da bridge di comunicazione inter-processo basato sul protocollo di trasporto TCP/IP, disaccoppiando l'ambiente grafico C# dal middleware robotico sottostante.

3.4.1 Architettura client-server

La comunicazione si struttura secondo un classico modello di rete client-server composto da due entità distinte:

- **ROS-TCP-Connector (Lato Unity - Client):** È un pacchetto integrato direttamente all'interno dell'ambiente Unity sotto forma di libreria C#. Opera come client TCP: una volta configurati l'indirizzo IP della macchina remota e la porta di ascolto (tipicamente la porta standard 10000), il connettore stabilisce una connessione socket persistente verso la workstation.
- **ROS-TCP-Endpoint (Lato ROS 2 - Server):** È un nodo ROS 2 (sviluppato in Python) in esecuzione sulla macchina host. Agisce come server TCP in ascolto su socket: accetta la connessione in ingresso dal visore, riceve i flussi di byte, li deserializza e si occupa di istanziare i corrispondenti publisher e subscriber all'interno del *computational graph* di ROS 2.

In questa architettura, la topologia di rete richiede esclusivamente che il visore AR e il PC su cui gira ROS 2 siano raggiungibili all'interno della medesima sottorete (es. tramite connessione Wi-Fi locale), rendendo il deployment flessibile e privo di vincoli hardware complessi.

3.4.2 Serializzazione e generazione dei messaggi

Per consentire lo scambio di dati tra Unity e ROS 2, è necessario che le informazioni vengano strutturate secondo il formato standard dei messaggi di ROS 2 (definiti nei file `.msg`, come ad esempio `sensor_msgs/Image` per i frame visivi o `geometry_msgs/PoseStamped` per le coordinate spaziali). Il pacchetto ROS-TCP-Connector integra uno strumento di generazione automatica (ROS Message Generation) che converte le definizioni dei messaggi ROS in corrispondenti classi C# utilizzabili all'interno dell'ambiente Unity. Il funzionamento generale del flusso di comunicazione si articola nei seguenti passaggi:

- Incapsulamento e serializzazione: All'interno di Unity i dati applicativi vengono inseriti direttamente nelle istanze delle classi C# generate. Tali strutture dati vengono quindi serializzate in un flusso binario di byte.
- Trasmissione via socket: Il client TCP integrato in Unity invia il pacchetto dati attraverso la connessione di rete verso la macchina host.
- Ricezione e pubblicazione: Il nodo server ROS-TCP-Endpoint in esecuzione su ROS 2 riceve i byte dal socket, li deserializza e si occupa di pubblicare effettivamente il messaggio sul corrispondente topic del grafo computazionale. Lo stesso meccanismo opera in modo simmetrico nel caso in cui Unity debba sottoscrivere a topic pubblicati da nodi ROS 2.

3.5 ORB-SLAM3

3.5.1 Algoritmi di SLAM (Simultaneous Localization and Mapping)

L'acronimo SLAM (Simultaneous Localization and Mapping) identifica una classe di algoritmi e tecniche computazionali che consentono a un dispositivo autonomo (come un robot mobile o un visore per la realtà aumentata) di costruire una mappa di un ambiente sconosciuto e, contestualmente, stimare la propria posizione e orientamento (posa a 6 DoF) all'interno di tale mappa in tempo reale. Nei sistemi di Realtà Aumentata e Spatial Computing, lo SLAM rappresenta la tecnologia abilitante fondamentale: senza di esso, il visore non potrebbe comprendere la geometria dell'ambiente circostante né calcolare gli spostamenti dell'utente, rendendo impossibile ancorare stabilmente elementi digitali nello spazio fisico. Da un punto di vista operativo, l'algoritmo elabora in tempo reale il flusso continuo di dati grezzi provenienti dai sensori di bordo (come telecamere, telecamere stereoscopiche, sensori di profondità o unità inerziali IMU). In una prima fase di elaborazione visiva (front-end), il sistema individua ed estrae punti di riferimento caratteristici dall'immagine (detti feature o punti di interesse) e li confronta tra fotogrammi consecutivi, stimando lo spostamento relativo istante per istante tramite tecniche di odometria visiva. Questa stima locale, tuttavia, è inevitabilmente soggetta a una progressiva accumulazione di errori nel tempo, nota come "drift". Per garantire consistenza globale alla ricostruzione, subentra la fase di ottimizzazione (back-end), che modella le relazioni spaziali tra le pose della camera e i punti tridimensionali dell'ambiente, minimizzando l'errore riproiettivo complessivo mediante tecniche di ottimizzazione non lineare (*Bundle Adjustment*). Il processo si completa con il riconoscimento dei luoghi e la chiusura dei cicli (loop closure): qualora il dispositivo ritorni in un'area precedentemente esplorata, il sistema riconosce la corrispondenza visiva, cor-

regge l'errore geometrico accumulato lungo l'intera traiettoria e allinea coerentemente la mappa globale.

3.5.2 ORB-SLAM3

Tra le architetture SLAM basate su visione computazionale, ORB-SLAM3 [4] rappresenta uno dei sistemi open-source più avanzati e completi dello stato dell'arte. Si tratta di una libreria versatile in grado di eseguire SLAM visivo, visivo-inerziale e multi-mappa in tempo reale su CPU convenzionali, garantendo elevate prestazioni senza la stretta necessità di accelerazione hardware dedicata. Il punto di forza dell'architettura risiede nell'impiego uniforme delle feature ORB (*Oriented FAST and Rotated BRIEF*) in tutti i moduli del sistema, dall'inseguimento rapido fotogramma per fotogramma alla fase di riconoscimento dei luoghi. Le feature sono dettagli visivi ben riconoscibili (come angoli o spigoli) che la telecamera usa come punti di riferimento. Il metodo ORB si occupa di individuare questi punti nell'immagine in modo rapidissimo e di creare per ciascuno un descrittore binario, ovvero una sequenza compatta di bit che ne codifica l'aspetto circostante rendendo il riconoscimento molto leggero. In questo modo il computer può confrontare centinaia di punti tra un fotogramma e l'altro quasi istantaneamente, riconoscendoli anche se la telecamera si sposta o ruota. Questa scelta assicura un'eccellente invarianza ai cambi di punto di vista e alle variazioni di illuminazione, mantenendo un costo computazionale estremamente contenuto. Rispetto alle iterazioni precedenti della libreria, ORB-SLAM3 introduce un'integrazione stretta con i sensori inerziali, permettendo di mantenere un tracciamento robusto anche in scenari critici caratterizzati da movimenti rapidi della camera o temporanee occlusioni visive. Un'ulteriore innovazione chiave è costituita dal sistema Atlas, un motore di gestione multi-mappa che consente al sistema di operare senza interruzioni anche in caso di perdita del tracciamento (tracking loss). Quando il tracciamento visivo fallisce, ORB-SLAM3 non interrompe l'esecuzione ma inizializza una nuova sottomappa locale; non appena il dispositivo riconosce un'area mappata in precedenza, l'algoritmo esegue un'operazione di fusione seamlessly unificando le sottomappe disconnesse in un unico modello globale coerente. Grazie a questa combinazione di robustezza, accuratezza metrica e flessibilità nell'adattarsi a diverse configurazioni di sensori (monoculari, stereoscopiche e RGB-D), ORB-SLAM3 è uno degli algoritmi di SLAM più utilizzati.

Capitolo 4

Progettazione e implementazione del framework

4.1 Requisiti funzionali e vincoli

La progettazione dell'architettura di comunicazione tra il visore Magic Leap 2 e l'ecosistema ROS 2 ha richiesto un'attenta analisi preliminare sia degli obiettivi applicativi, sia delle limitazioni hardware e computazionali tipiche dei dispositivi embedded e della trasmissione wireless. Di seguito vengono formalizzati i requisiti funzionali individuati e i vincoli che hanno guidato lo sviluppo del software.

4.1.1 Requisiti funzionali

I requisiti funzionali definiscono le capacità operative e i servizi che il bridge software sviluppato a bordo del visore deve garantire:

- **Acquisizione multi-sensoriale:** Il sistema deve interfacciarsi con l'hardware del visore per estrarre flussi eterogenei di dati. Nello specifico, deve essere in grado di acquisire i flussi video (RGB e telecamere di tracking ambientale), la stima della posa spaziale nello spazio 3D calcolata dall'headset e le letture dei sensori ausiliari (come IMU e sensore di luminosità ambientale).
- **Serializzazione e trasmissione dati su ROS 2:** Le informazioni sensoriali estratte devono essere impacchettate e serializzate rispettando i messaggi standard di ROS 2 (in particolare la suite `sensor_msgs` e i relativi tipi per immagini, pose e letture grezze). Per ogni flusso video, il bridge deve garantire la pubblicazione sincronizzata del relativo messaggio di calibrazione intrinseca (`CameraInfo`), permettendo la corretta interpretazione delle immagini da parte dei nodi riceventi.

- Configurazione dinamica a runtime: Per massimizzare la flessibilità operativa ed evitare continui cicli di ricompilazione del package Unity/Android su visore, il sistema deve consentire la configurazione dei parametri di streaming (quali sensori abilitare, risoluzioni e framerate desiderati) direttamente dal lato ROS 2 tramite un file di configurazione `config.yaml` letto all'avvio dell'applicazione.

4.1.2 Vincoli di progetto e di sistema

I vincoli architetturali e tecnologici hanno indirizzato le scelte implementative per garantire massima compatibilità, efficienza e scalabilità:

- Standardizzazione e interoperabilità: Per consentire l'integrazione immediata dei dati del visore con algoritmi terzi della comunità robotica (come pipeline di visual SLAM o nodi di navigazione), l'architettura dei topic e l'albero delle trasformazioni geometriche (TF) sono stati allineati agli standard de facto utilizzati dai principali sensori di visione robotica, in primis le videocamere RGB-D della famiglia Intel RealSense. Ciò ha comportato l'adozione rigorosa delle convenzioni di orientamento per i frame ottici (`_optical_frame`), garantendo che il sistema di riferimento delle telecamere rispetti la convenzione ottica standard e una nomenclatura dei topic simili.
- Efficienza di rete e gestione delle Risorse: Lo streaming di molteplici flussi sensoriali e di frame video ad alta risoluzione tramite rete introduce un potenziale collo di bottiglia sulla banda disponibile. Il sistema è stato quindi vincolato all'ottimizzazione del payload trasmesso e alla minimizzazione dell'overhead di serializzazione del protocollo TCP, garantendo frequenze di aggiornamento adeguate senza saturare la rete locale o appesantire la CPU del visore.
- Modularità ed estensibilità del software: L'applicazione su Unity è stata concepita seguendo un pattern a componenti disaccoppiati (script indipendenti per ciascuna tipologia di sensore). Questo vincolo progettuale assicura la manutenibilità del codice e consente di integrare agevolmente in futuro nuovi flussi sensoriali (come eye tracking o hand tracking) senza dover riscrivere la logica di connessione o interferire con i moduli già collaudati.

4.2 Architettura del framework e negoziazione dei parametri

Per garantire un disaccoppiamento efficace tra la logica di controllo su ROS 2 e l'esecuzione a bordo del visore, il sistema è stato progettato secondo un modello modulare che inizia con una fase iniziale di negoziazione e configurazione dinamica dei parametri della trasmissione. L'infrastruttura si compone di due nodi sul lato ROS 2 (l'endpoint di comunicazione e il nodo di configurazione) e di un insieme coordinato di script a bordo dell'applicazione Unity.

4.2.1 Componenti del sistema e flusso di negoziazione

Il funzionamento del bridge si basa su un meccanismo di sincronizzazione a stati tra ROS 2 e Unity:

1. **Avvio dell'infrastruttura ROS 2:** Vengono inizializzati l'endpoint di rete (ROS-TCP-Endpoint), incaricato di gestire la connessione a basso livello con Unity, e il nodo di configurazione. Quest'ultimo carica in memoria i parametri operativi definiti all'interno di un file `config.yaml` e si mette in ascolto sul topic dedicato alle richieste provenienti dal visore.
2. **Richiesta di configurazione (handshake):** All'avvio dell'applicazione su Magic Leap 2, lo script coordinatore (Configurator) invia un messaggio di sincronizzazione su un topic di richiesta (`/request`).
3. **Serializzazione e Invio dei Parametri:** Ricevuta la richiesta, il nodo di configurazione serializza i parametri letti dal file YAML in una stringa strutturata (formato JSON) e la pubblica su un apposito topic di risposta destinato al visore (`/magic_leap2/config`).
4. **Parsing e avvio selettivo dei flussi:** Il Configurator a bordo di Unity riceve il payload JSON e ne effettua il parsing. La struttura modulare dell'applicazione prevede uno script (modulo) per la gestione di ogni sensore o funzionalità. Questi script partono a priori tutti insieme ma prima di avviare la lettura e l'invio dei messaggi a ROS 2 aspettano che il Configurator assegni loro i rispettivi parametri (abilitazione, risoluzioni, framerate) alle variabili interne.
5. **Attivazione dei sensori:** Completata l'assegnazione dei parametri, il Configurator imposta a true i flag di avvio (`enabled`) dei soli moduli attivati. Tale transizione sblocca l'esecuzione delle coroutine di streaming all'interno dei rispettivi script, dando inizio alla pubblicazione continua dei dati sensoriali.

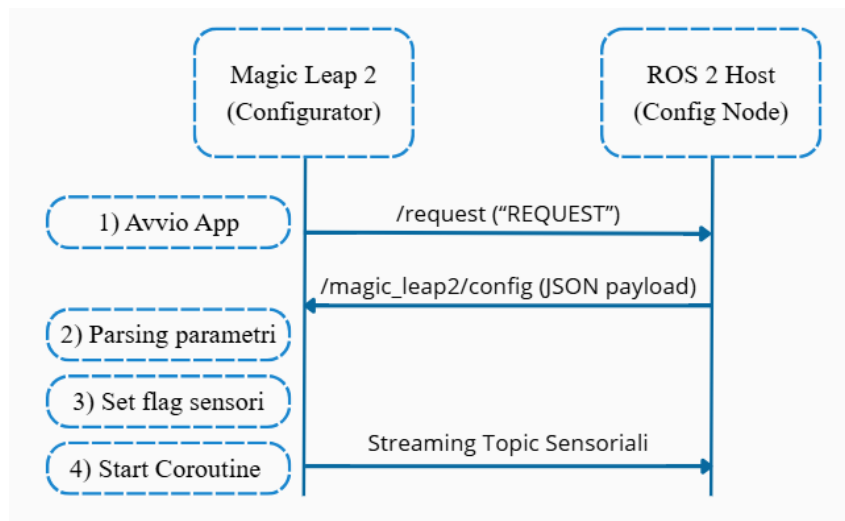


Figura 4.1: Flusso di negoziazione dei parametri della trasmissione tra il Magic Leap 2 e ROS 2. (Fonte: elaborazione dell'autore)

4.2.2 Struttura del file di configurazione (`config.yaml`)

La configurazione centralizzata consente all'utente di definire in anticipo il carico computazionale e l'occupazione di banda della rete senza dover modificare o ricompilare il codice su Unity o Android. Tramite il file `config.yaml` è possibile specificare:

- **Prefisso dei topic del visore:** Di default i topic su cui pubblica il visore sono `/magic_leap2/...` ma è possibile modificare il nome a proprio piacimento.
- **Abilitazione dei sensori:** Flag booleani per attivare o disattivare selettivamente ciascun componente (telecamere di contesto, fotocamera RGB, sensore ToF, IMU e sensore di luminosità).
- **Stream (dove supportato dal visore):** Il visore prevede che alcuni sensori abbiano più stream possibili che fanno cose completamente diverse come ad esempio la camera `picture_center` ha uno stream per le immagini e un altro che mostra la mixed reality ovvero le immagini della telecamera sovrapposte a ciò che proietta il visore.
- **Risoluzione e formato delle immagini (dove supportato dal visore):** Dimensioni in pixel e codifica video per ciascuna camera abilitata, in funzione delle esigenze dell'algoritmo ricevente (ad esempio per bilanciare accuratezza visiva e carico di rete).
- **Frequenze di pubblicazione (dove supportato dal visore):** Parametri di framerate per scandire la periodicità di invocazione delle routine di invio dei frame e dei dati ambientali.

A seguire è riportato un esempio di file di configurazione, `config.yaml`:

```

/config_ML_node:
  ros__parameters:

    general:
      topic_name : "/magic_leap2"

    picture_center:
      enabled: false
      # stream 0 is the simple RGB camera stream.
      # stream 1 is the AR stream with the world as background and the user as
      # foreground.
      stream: 0
      # for stream 0 the possible resolution are 640x480, 1280x720, 1920x1080,
      # 3840x2160, 2048x1536, 1280x960, 1440x1080, 2880x2160, 4096x3072.
      # for stream 1 the possible resolution are 648x720, 972x1080, 1944x2160,
      # 960x720, 1440x1080, 2880x2160.
      resolution_width: 1920
      resolution_height: 1080
      update_rate: 30 # possible values in range [30, 60]
      format: "jpeg"

    depth_center:
      enabled: false
      # stream 0 is the long range depth stream.
      # stream 1 is the short range depth stream.
      stream: 1
      resolution_width: 544
      resolution_height: 480
      # for stream 0 the possible update rates are 1, 5.
      # for stream 1 the possible update rates are 5, 30, 60.
      update_rate: 30
      format: "Depth32" # possible values = "Depth32", "DepthRaw".

    world_center:
      enabled: false
      stream: 0 # possible values = 0, 1
      resolution_width: 1016

```

```

resolution_height: 1016
update_rate: 30
format: "Grayscale"

world_right:
  enabled: false
  stream: 0
  resolution_width: 1016
  resolution_height: 1016
  update_rate: 30
  format: "Grayscale"
# possible values = 0, 1

world_left:
  enabled: false
  stream: 0
  resolution_width: 1016
  resolution_height: 1016
  update_rate: 30
  format: "Grayscale"
# possible values = 0, 1

eye_temple_right:
  enabled: false
  stream: 0
  resolution_width: 400
  resolution_height: 400
  update_rate: 30
  format: "Grayscale"

eye_temple_left:
  enabled: false
  stream: 0
  resolution_width: 400
  resolution_height: 400
  update_rate: 30
  format: "Grayscale"

eye_nasal_right:

```

```

    enabled: false
    stream: 0
    resolution_width: 400
    resolution_height: 400
    update_rate: 30
    format: "Grayscale"

eye_nasal_left:
    enabled: false
    stream: 0
    resolution_width: 400
    resolution_height: 400
    update_rate: 30
    format: "Grayscale"

# the other sensors include gyroscope, accelerometer and
# ambient light sensor.
other_sensors:
    enabled: false

```

4.2.3 Architettura a componenti e modello di esecuzione in Unity

All'interno dell'ambiente Unity, l'applicazione è strutturata attorno a un unico GameObject centrale a cui sono associati script C# dedicati e indipendenti per ciascuna tipologia di flusso sensoriale (telecamere, sensori inerziali e di luminosità). Tale organizzazione garantisce una forte modularità e isolamento tra i processi di acquisizione. Ciascun modulo implementa un modello di esecuzione comune basato sulle seguenti fasi:

- Inizializzazione (Start): Configura le strutture dati interne e i publisher di ROS 2, mantenendo una variabile booleana di abilitazione inizialmente impostata a false.
- Sblocco e controllo del flusso (Update): A ogni frame di rendering, il metodo Update monitora lo stato del flag di abilitazione. Non appena lo script di configurazione imposta il flag a true, viene avviata una coroutine asincrona dedicata all'acquisizione dei frame e alla pubblicazione dei messaggi `sensor_msgs/Image` e `sensor_msgs/CameraInfo` con il relativo timestep.
- Aggiornamento continuo delle pose: Parallelamente all'invio dei frame, il ciclo Update continua a estrarre la posa 6DoF aggiornata del sensore (espressa rispetto al frame ottico di

riferimento) e a pubblicarla su ROS 2. Questa separazione consente di mantenere l'albero delle trasformazioni geometriche (tf) aggiornato ad alta frequenza, senza vincolarlo ai frame acquisiti dalle telecamere.

- Integrazione di sensori a basso livello: Per i flussi non gestiti nativamente tramite le API standard di Unity (come i sensori IMU e di luminosità), lo script dedicato interroga a runtime un plugin nativo sviluppato in Kotlin per Android OS, esponendo le letture verso l'infrastruttura di pubblicazione secondo il medesimo paradigma di sincronizzazione.

4.3 Implementazione e caratterizzazione dei moduli sensoriali

Per consentire una percezione completa dell'ambiente e delle dinamiche dell'utente, l'applicazione sviluppata acquisisce l'intero set di flussi sensoriali messi a disposizione dall'hardware del visore Magic Leap 2 e li pubblica in tempo reale su appositi topic ROS 2. Nello specifico, il bridge gestisce:

- Telecamere di tracciamento ambientale (World Cameras): tre flussi monocromatici (in scala di grigi) a basso livello, completi di calibrazione intrinseca e pose spaziali;
- Telecamera frontale RGB (Picture Center): immagini ad alta definizione compresse in formato JPEG, disponibili sia come flusso video ottico standard sia come visualizzazione in realtà mista con elementi olografici sovrapposti;
- Sensore di profondità (Depth Camera): dati a tempo di volo (ToF) configurabili per la misura metrica delle distanze (Depth32) o per la cattura dell'intensità infrarossa riflessa (DepthRaw);
- Telecamere per l'Eye Tracking: quattro stream a infrarossi dedicati all'analisi dello sguardo e dei movimenti oculari;
- Sensori di bordo ausiliari (IMU e luminosità): accelerazioni lineari, velocità angolari e livello di illuminamento esterno, estratti con timestamp hardware tramite un plugin nativo Android.

Nelle sottosezioni seguenti vengono descritte dettagliatamente l'implementazione, la struttura dei topic, i formati dei messaggi, le trasformazioni spaziali (TF Tree) e le relative caratteristiche operative di ciascun modulo sensoriale.

4.3.1 Telecamere di tracciamento ambientale (World Cameras)



Figura 4.2: Esempio di acquisizione dei fotogrammi in scala di grigi provenienti dalle tre telecamere di tracciamento ambientale (World Cameras): da sinistra verso destra, *World Left*, *World Center* e *World Right*.

Descrizione e funzionamento

Il visore integra tre telecamere monocromatiche ad ampio campo visivo (FOV, Field of View), ottimizzate a bassa latenza per l'analisi spaziale e la comprensione dell'ambiente circostante. Come descritto nella Sezione 3.1.2, i sensori sono disposti frontalmente e lateralmente nel visore e identificati convenzionalmente come:

- World Center: telecamera centrale;
- World Left: telecamera laterale sinistra;
- World Right: telecamera laterale destra.

Ciascun sensore acquisisce immagini in scala di grigi con una risoluzione fissa di 1016×1016 pixel a una frequenza di campionamento di 30 Hz. Considerando una profondità di colore pari a 8 bit per pixel, il flusso dati grezzo generato da una singola telecamera comporta un throughput di circa 250 Mbit/s.

Topic ROS 2 e messaggi

Ciascuna telecamera può essere attivata o disattivata in modo indipendente. I topic e le relative trasformazioni geometriche vengono registrati e pubblicati esclusivamente se il corrispondente modulo sensoriale risulta abilitato. Per evitare duplicazioni, la struttura dei topic per una generica camera `<name>` (con `<name> ∈ [ world_center, world_left, world_right ]`) è così definita:

Topic	Tipo di Messaggio	Descrizione
/magic_leap2/<name>/image_raw	sensor_msgs/msg/Image	Flusso grezzo delle immagini acquisite.
/magic_leap2/<name>/camera_info	sensor_msgs/msg/CameraInfo	Parametri intrinseci e modello di calibrazione della telecamera.
/magic_leap2/<name>/metadata	std_msgs/msg/String	Metadati del sensore (es. guadagno analogico/digitale, tempo di esposizione).

Trasformazioni spaziali (TF Tree)

Il bridge pubblica le pose delle telecamere sul topic standard `/tf` a una frequenza di 60 Hz, espresse rispetto al sistema di coordinate fisso `magic_leap_world` (origine definita all'avvio dell'applicazione). Per ciascun sensore abilitato vengono pubblicati due specifici frame di coordinate per garantire piena compatibilità sia con gli standard robotici che con le librerie di computer vision:

- `<name>_frame`: orientato secondo la convenzione standard di ROS (asse X in avanti, Y a sinistra, Z verso l'alto);
- `<name>_optical_frame`: orientato secondo la convenzione ottica standard (asse Z lungo l'asse ottico in avanti, X a destra, Y verso il basso).

Limitazioni ed eventuali criticità

La caratterizzazione sperimentale delle tre telecamere di tracciamento ambientale ha evidenziato prestazioni ottimali: il flusso video mantiene una frequenza di campionamento rigorosamente stabile a 30 Hz, con un livello di jitter trascurabile e un'ottima reattività nei cambi di luminosità. L'unica reale criticità è legata all'impatto sul canale di trasmissione: trattandosi di stream non compressi ad alta risoluzione (1016×1016 pixel a 8 bit), l'attivazione contemporanea di tutti e tre i moduli genera un carico di rete complessivo pari a circa 750 Mbit/s. Tale volume di dati può portare alla saturazione della larghezza di banda disponibile su reti Wi-Fi convenzionali, rendendo consigliabile l'abilitazione selettiva della sola telecamera di interesse (es. la sola World Center) tramite il file `config.yaml` o l'impiego di una connessione cablata dedicata.

4.3.2 Telecamera frontale RGB ad alta risoluzione (Picture Center)

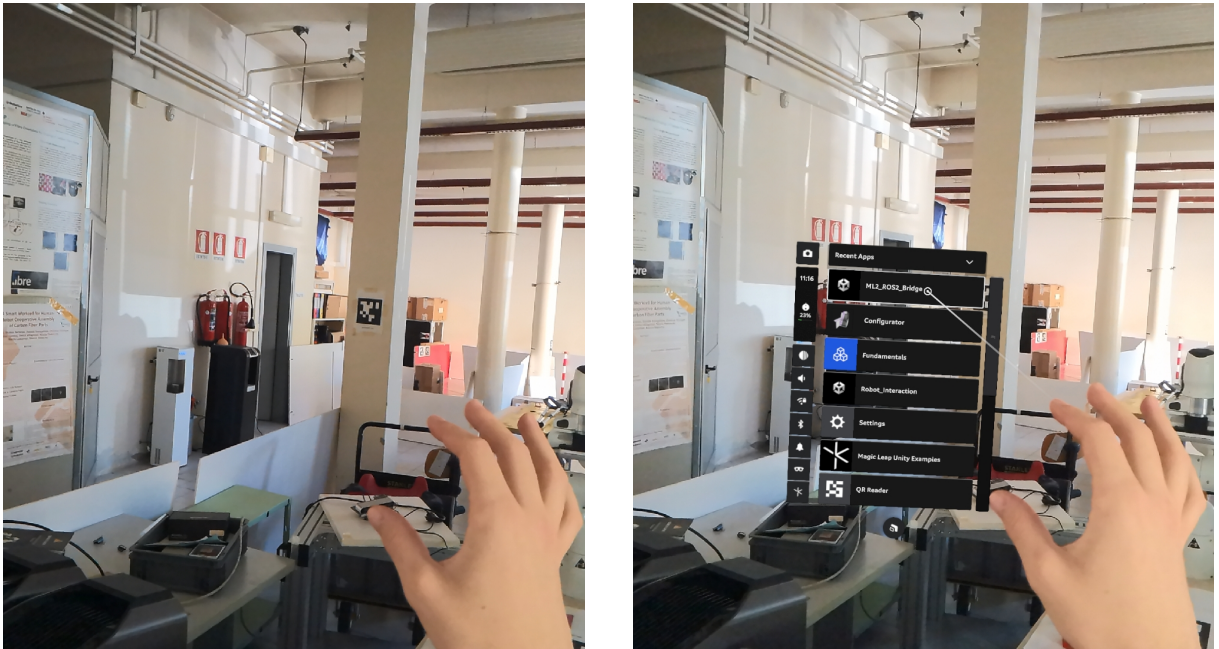


Figura 4.3: Flussi acquisiti dalla telecamera frontale RGB (*Picture Center*): a sinistra lo stream video convenzionale (*Stream 0*), a destra lo stream in realtà mista (*Stream 1*) con sovrapposizione degli elementi olografici dell'interfaccia utente.

Descrizione e funzionamento

Posizionata frontalmente al centro del visore (denominata convenzionalmente *Picture Center*), questa telecamera ad alta risoluzione è concepita per l'acquisizione di dettagli visivi, fotografie e registrazioni ad alta definizione. Il flusso di acquisizione converte e comprime nativamente i frame in formato JPEG direttamente a bordo del dispositivo per ottimizzare il payload di rete. Le API del visore mettono a disposizione due distinte modalità di streaming:

- Stream 0 (RGB Standard Stream): Flusso video ottico convenzionale della camera RGB.
- Stream 1 (Mixed Reality / AR Stream): Flusso sintetico composito in cui al video reale dell'ambiente circostante vengono sovrapposti gli elementi grafici e gli ologrammi visualizzati dall'utente nel visore.

Entrambe le modalità consentono di selezionare frequenze di campionamento nominali di 30 Hz o 60 Hz e supportano un ampio spettro di risoluzioni:

- Risoluzioni stream 0: 640×480 , 1280×720 , 1920×1080 , 3840×2160 , 2048×1536 , 1280×960 , 1440×1080 , 2880×2160 , 4096×3072 pixel.

- Risoluzioni stream 1: 648×720 , 972×1080 , 1944×2160 , 960×720 , 1440×1080 , 2880×2160 pixel.

Grazie alla compressione JPEG integrata nel payload, l'occupazione di banda risulta significativamente contenuta rispetto a un flusso grezzo non compresso, variando in funzione della risoluzione e del framerate selezionati.

Topic ROS 2 e messaggi

A seconda della modalità selezionata nel file di configurazione (`config.yaml`), il bridge registra e pubblica i dati sui relativi topic ROS 2:

Topic per la modalità RGB standard (Stream 0):

Topic	Tipo di Messaggio	Descrizione
<code>/magic_leap2/color/image_raw/compressed</code>	<code>sensor_msgs/msg/CompressedImage</code>	Flusso delle immagini compresse acquisite.
<code>/magic_leap2/color/camera_info</code>	<code>sensor_msgs/msg/CameraInfo</code>	Parametri intrinseci e modello di calibrazione della telecamera.
<code>/magic_leap2/color/metadata</code>	<code>std_msgs/msg/String</code>	Metadati del sensore (es. guadagno analogico/digitale, tempo di esposizione).

Topic per la modalità Mixed Reality (Stream 1):

Topic	Tipo di Messaggio	Descrizione
<code>/magic_leap2/mixed_reality/image_raw/compressed</code>	<code>sensor_msgs/msg/CompressedImage</code>	Flusso delle immagini compresse acquisite.
<code>/magic_leap2/mixed_reality/camera_info</code>	<code>sensor_msgs/msg/CameraInfo</code>	Parametri intrinseci e modello di calibrazione della telecamera.
<code>/magic_leap2/mixed_reality/metadata</code>	<code>std_msgs/msg/String</code>	Metadati del sensore (es. guadagno analogico/digitale, tempo di esposizione).

Trasformazioni spaziali (TF Tree)

Il bridge pubblica le pose della telecamera sul topic standard `/tf` a una frequenza di 60 Hz, espresse rispetto al sistema di coordinate fisso `magicleap_world` (origine definita all'avvio dell'applicazione). Vengono pubblicati due specifici frame di coordinate:

- `color_frame`: orientato secondo la convenzione standard di ROS (asse *X* in avanti, *Y* a sinistra, *Z* verso l'alto);

- `color_optical_frame`: orientato secondo la convenzione ottica standard (asse Z lungo l'asse ottico in avanti, X a destra, Y verso il basso).

Limitazioni ed eventuali criticità

La caratterizzazione sperimentale del modulo ha evidenziato alcune criticità operative legate principalmente alla gestione termica del dispositivo:

- **Overheating e thermal throttling**: L'acquisizione ad alta risoluzione combinata con l'onere di compressione JPEG continuo genera un rapido incremento della temperatura interna del visore.
- **Degrado del framerate**: Impostando una frequenza nominale di 60 Hz, il sensore raramente riesce a sostenere tale regime all'avvio, assestandosi inizialmente su frequenze inferiori. All'aumentare della temperatura, subentra un fenomeno di thermal throttling che degrada progressivamente il framerate effettivo nell'arco di una decina di minuti.

Per tali ragioni, il modulo risulta poco adatto ad applicazioni che richiedono flussi continui ad alta frequenza (come la visual odometry in tempo reale), prestandosi invece in modo ottimale a compiti di ispezione visuale a intervalli periodici, scatto di fotogrammi statici ad altissima risoluzione o registrazione di brevi sequenze in realtà mista per la supervisione dell'operatore.

4.3.3 Sensore di profondità (Depth camera)

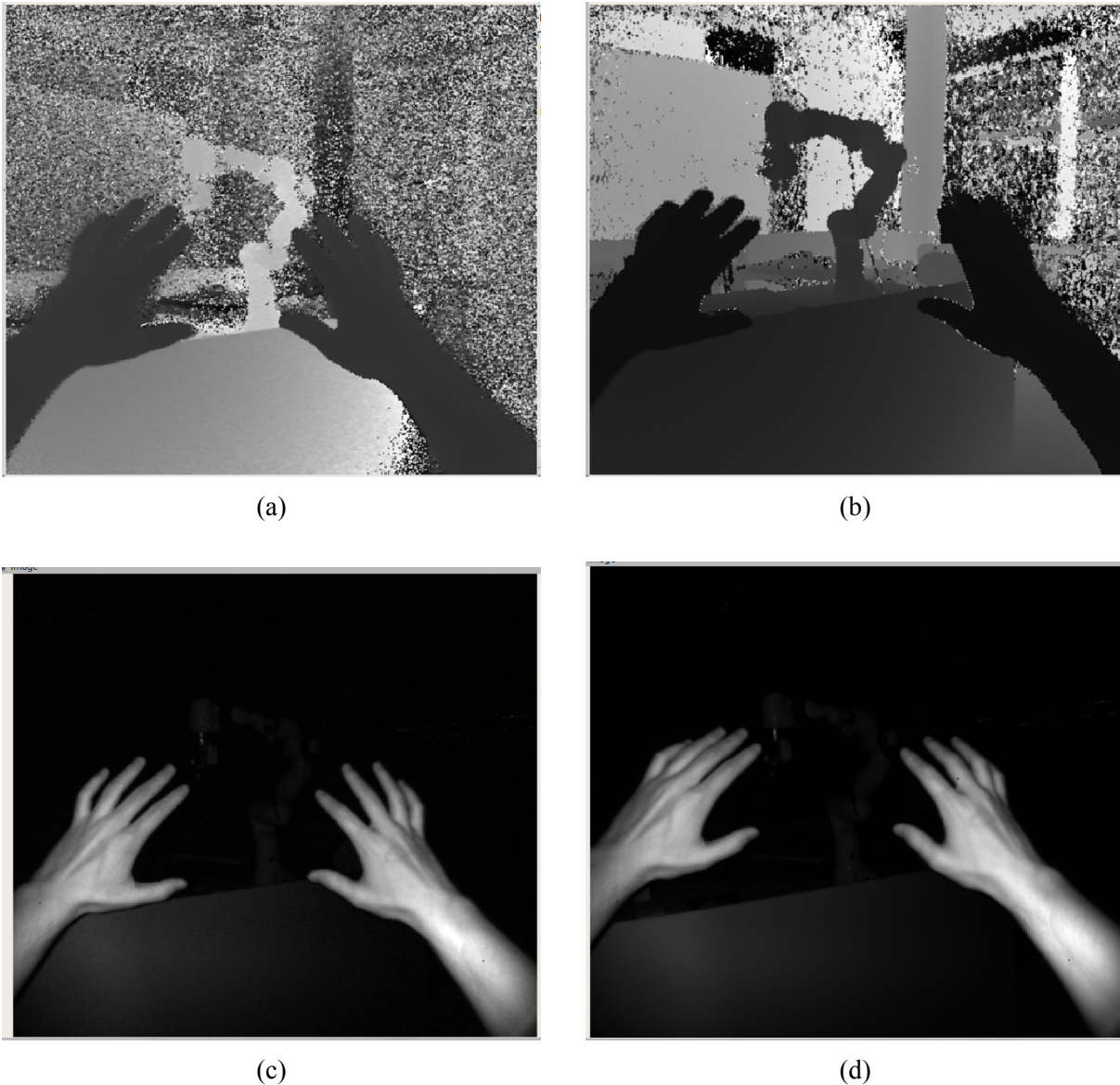


Figura 4.4: Confronto visivo tra le quattro modalità operative del sensore di profondità Time-of-Flight (ToF): (a) Short Range con codifica metrica *Depth32*; (b) Long Range con codifica metrica *Depth32*; (c) Short Range con intensità infrarossa grezza *DepthRaw*; (d) Long Range con intensità infrarossa grezza *DepthRaw*.

Descrizione e funzionamento

Posizionato frontalmente (denominato convenzionalmente Depth Center), il sensore di profondità si basa su tecnologia a tempo di volo (Time-of-Flight, ToF) per ricostruire la geometria tridimensionale dell'ambiente circostante. Il sensore opera a una risoluzione fissa di 544×480 pixel e fornisce i dati secondo due distinti formati di output:

- **Depth32 (Mappa di Profondità):** Matrice di valori in virgola mobile a 32 bit (`float32`) in cui ciascun elemento rappresenta la distanza metrica stimata del punto rispetto al piano focale del sensore.
- **DepthRaw (Intensità Infrarossa):** Matrice di valori a 32 bit (`float32`) contenente la quantità grezza di luce infrarossa riflessa e catturata dal ricevitore, particolarmente indicata per applicazioni di motion capture o riconoscimento di marker retro-riflettenti.

Le API del visore mettono a disposizione due modalità operative di streaming:

- **Stream 0 (Long Range):** Configurazione ottimizzata per la mappatura ambientale a medio-lungo raggio (distanze comprese tra 0.5 m e 5.0 m) con accuratezza centimetrica. Supporta frequenze di campionamento configurabili pari a 1 Hz e 5 Hz.
- **Stream 1 (Short Range):** Configurazione ad alta precisione millimetrica per distanze ravvicinate (tra 0.2 m e 1.2 m), ideale per l'interazione fine, il tracciamento delle mani (hand tracking) e la stima delle gesture. Supporta frequenze di campionamento selezionabili pari a 5 Hz e 30 Hz.

Considerando una risoluzione di 544×480 pixel e un payload di 4 byte per pixel (formato `float32`), il throughput teorico non compresso generato dal sensore risulta pari a circa:

- ≈ 8.35 Mbit/s a 1 Hz
- ≈ 41.78 Mbit/s a 5 Hz;
- ≈ 250.68 Mbit/s a 30 Hz.

Topic ROS 2 e messaggi

In base al formato dati selezionato nel file `config.yaml`, il bridge provvede a registrare e pubblicare le informazioni sui relativi topic:

Topic per il formato di profondità metrica (Depth32):

Topic	Tipo di Messaggio	Descrizione
<code>/magic_leap2/depth/image_raw</code>	<code>sensor_msgs/msg/Image</code>	Matrice grezza dei valori di profondità metrica (codifica 32FC1).
<code>/magic_leap2/depth/camera_info</code>	<code>sensor_msgs/msg/CameraInfo</code>	Parametri intrinseci e modello di calibrazione del sensoreToF.
<code>/magic_leap2/depth/metadata</code>	<code>std_msgs/msg/String</code>	Metadati operativi (tempi di integrazione, modalità di illuminazione IR).

Topic per il formato a intensità infrarossa (DepthRaw):

Topic	Tipo di Messaggio	Descrizione
/magic_leap2/infra/ image_raw	sensor_msgs/ msg/Image	Matrice grezza dell'intensità di riflessione IR (codifica 32FC1).
/magic_leap2/infra/ camera_info	sensor_msgs/ msg/CameraInfo	Parametri intrinseci e modello di calibrazione del sensore.
/magic_leap2/infra/ metadata	std_msgs/ msg/String	Metadati del ricevitore IR.

Trasformazioni spaziali (TF Tree)

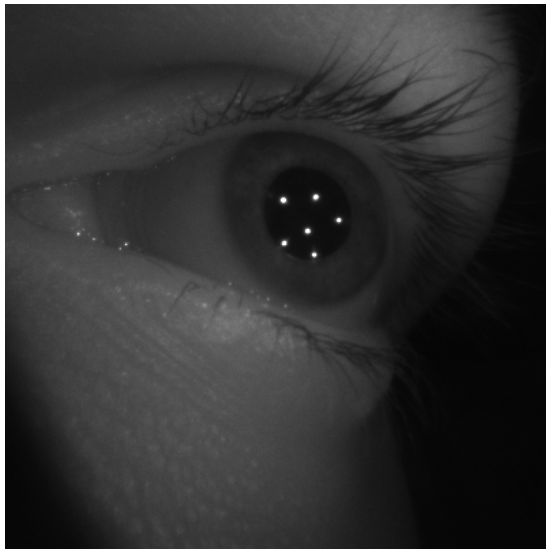
Il bridge pubblica le pose del sensore ToF sul topic `/tf` a una frequenza di 60 Hz rispetto al frame di riferimento `magic Leap world`:

- `depth_frame`: orientato secondo la convenzione standard di ROS (asse X in avanti, Y a sinistra, Z verso l'alto);
- `depth_optical_frame`: orientato secondo la convenzione ottica standard (asse Z lungo l'asse ottico in avanti, X a destra, Y verso il basso).

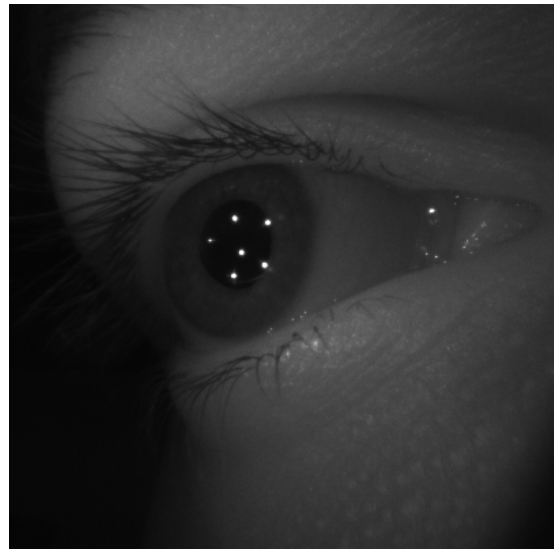
Limitazioni ed eventuali criticità

La validazione sperimentale ha evidenziato un comportamento nominale robusto, con latenze contenute e un throughput costante privo di oscillazioni. Tuttavia, è emersa un'incongruenza a livello di API per quanto concerne la modalità Short Range (Stream 1): sebbene le primitive di interrogazione del visore indichino come selezionabile un framerate pari a 60 Hz (e l'applicazione accetti tale parametro senza generare eccezioni o interruzioni del flusso), la frequenza effettiva di pubblicazione dei frame rimane vincolata a un limite hardware/firmware massimo pari a 30 Hz.

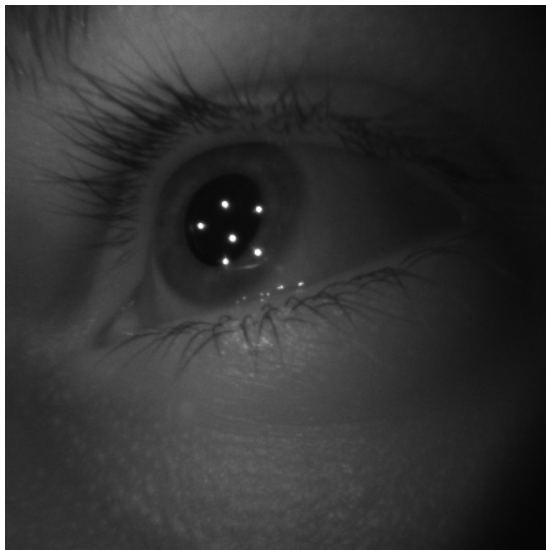
4.3.4 Telecamere per l'Eye Tracking e stima dello sguardo



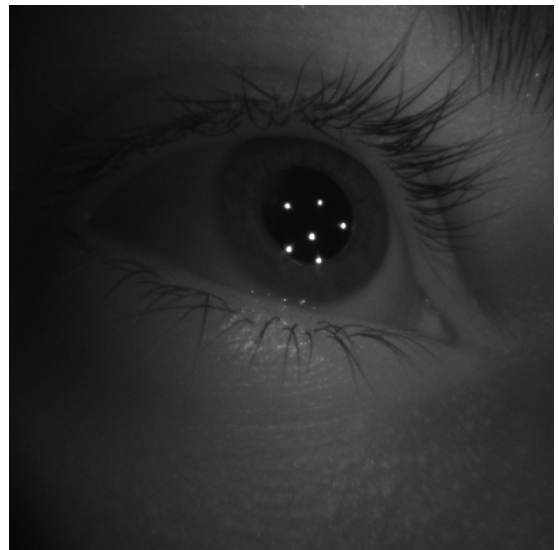
(a)



(b)



(c)



(d)

Figura 4.5: Flussi a infrarossi acquisiti dalle quattro telecamere dedicate all'Eye Tracking: (a) *Eye Nasal Right*, (b) *Eye Nasal Left*, (c) *Eye Temple Right* e (d) *Eye Temple Left*.

Descrizione e funzionamento

Il visore integra quattro telecamere monocromatiche a infrarossi specificamente orientate verso gli occhi dell'utente, dedicate al tracciamento oculare (eye tracking) e al riconoscimento della direzione dello sguardo (gaze recognition). I sensori sono disposti a coppie (uno sul lato nasale e uno sul lato temporale per ciascun occhio) e vengono identificati convenzionalmente come:

- `eye_nasal_right`: telecamera nasale per l'occhio destro;
- `eye_temple_right`: telecamera temporale per l'occhio destro;
- `eye_nasal_left`: telecamera nasale per l'occhio sinistro;
- `eye_temple_left`: telecamera temporale per l'occhio sinistro.

Ciascun sensore acquisisce immagini in scala di grigi con una risoluzione fissa di 400×400 pixel a una frequenza di campionamento pari a 30 Hz. Considerando una profondità di colore a 8 bit per pixel, il flusso di dati grezzo generato da una singola telecamera comporta un throughput teorico di circa 38.4Mbit/s. Complessivamente, nel caso di attivazione simultanea di tutti e quattro i sensori, il traffico di rete generato ammonta a circa 153.6 Mbit/s.

Topic ROS 2 e messaggi

Ciascuna telecamera può essere abilitata o disattivata in modo indipendente tramite il file di configurazione `config.yaml`. I topic vengono registrati e pubblicati esclusivamente se il corrispondente modulo risulta attivo. Per una generica telecamera `<name>` (con `<name>` $\in$ `{eye_nasal_right, eye_temple_right, eye_nasal_left, eye_temple_left}`), la struttura dei topic è definita come segue:

Topic	Tipo di Messaggio	Descrizione
<code>/magic_leap2/<name>/image_raw</code>	<code>sensor_msgs/msg/Image</code>	Flusso grezzo delle immagini monocromatiche acquisite (codifica mono8).
<code>/magic_leap2/<name>/metadata</code>	<code>std_msgs/msg/String</code>	Metadati del sensore (es. tempo di esposizione e guadagni analogico/digitale).

Limitazioni ed eventuali criticità

Durante la fase di sviluppo e caratterizzazione dei moduli di eye tracking sono emerse alcune limitazioni architetturali imposte dall'ecosistema software e dalle API del costruttore:

- Assenza di parametri intrinseci (CameraInfo): Le API di Magic Leap non espongono la matrice di calibrazione intrinseca né i coefficienti di distorsione ottica per le camere oculari, impedendo la generazione del topic standard `sensor_msgs/msg/CameraInfo`.
- Assenza di pose geometriche nel TF Tree: Il sistema operativo non fornisce le trasformazioni spaziali relative alla collocazione rigida delle quattro fotocamere rispetto al telaio centrale del visore; di conseguenza, non vengono pubblicati i frame di riferimento dedicati sull'albero `/tf`.

Nonostante tali vincoli informativi a livello di metadati spaziali, i quattro flussi video presentano una latenza ridotta e una frequenza di acquisizione estremamente stabile a 30 Hz, risultando pienamente utilizzabili per pipeline esterne di elaborazione delle immagini o algoritmi personalizzati di tracciamento pupillare.

4.3.5 Unità inerziali (IMU) e sensore di luminosità ambientale

Descrizione e funzionamento

L'architettura hardware del dispositivo include tre unità di misura inerziale (IMU) distinte e un sensore di luminosità ambientale:

- Headset IMU (Left e Right): Due IMU integrate direttamente sulla montatura degli occhiali, posizionate rispettivamente sul lato sinistro e destro del telaio per tracciare con precisione le dinamiche della testa.
- Compute pack IMU: Una terza IMU collocata all'interno dell'unità di elaborazione esterna (Compute Pack).
- Sensore di Luce Ambientale: Fotodiodo integrato sul visore per il monitoraggio dell'illuminamento esterno.

L'accesso a tali sensori tramite le API standard del Magic Leap SDK ha evidenziato alcune limitazioni critiche: in particolare, l'impossibilità di interrogare simultaneamente tutte e tre le IMU e l'assenza dei timestamp hardware originali associati all'istante esatto di acquisizione del campione. Per superare tale vincolo, è stato sviluppato un plugin nativo per Android in linguaggio Kotlin. Il plugin interagisce a basso livello con il servizio `SensorManager` del sistema operativo Magic Leap OS (basato su Android Open Source Project), registrando i relativi listener

per ciascun accelerometro, giroscopio e sensore di luminosità. A runtime, lo script C# su Unity interroga il plugin nativo ed estrae le letture grezze unitamente ai rispettivi timestamp hardware con precisione nanosecondale. Dato il payload estremamente compatto dei messaggi numerici scalari e vettoriali, l'impatto complessivo di questi flussi sulla larghezza di banda della rete è trascurabile (nell'ordine di poche centinaia di kbit/s).

Topic ROS 2 e messaggi

I dati inerziali e fotometrici vengono serializzati e pubblicati sui rispettivi topic ROS 2 standard appartenenti alla suite `sensor_msgs`:

Topic	Tipo di Messaggio	Descrizione
/magic_leap2/accel_left/imu	sensor_msgs/msg/Imu	Accelerazione lineare IMU sinistra (headset).
/magic_leap2/gyro_left/imu	sensor_msgs/msg/Imu	Velocità angolare IMU sinistra (headset).
/magic_leap2/accel_right/imu	sensor_msgs/msg/Imu	Accelerazione lineare IMU destra (headset).
/magic_leap2/gyro_right/imu	sensor_msgs/msg/Imu	Velocità angolare IMU destra (headset).
/magic_leap2/accel_compute_pack/imu	sensor_msgs/msg/Imu	Accelerazione lineare IMU Compute Pack.
/magic_leap2/gyro_compute_pack/imu	sensor_msgs/msg/Imu	Velocità angolare IMU Compute Pack.
/magic_leap2/light_sensor	sensor_msgs/msg/Illuminance	Livello di illuminamento ambientale misurato.

Limitazioni ed eventuali criticità

La frequenza di campionamento esposta dallo strato Android tramite `SensorManager` per le unità inerziali risulta limitata a circa 40 Hz, presumibilmente a causa di filtri di decimazione o aggregazione interna applicati dal layer HAL (*Hardware Abstraction Layer*) del sistema operativo. Nonostante tale frequenza sia inferiore rispetto ai regimi nominali massimi dei sensori inerziali (spesso superiori a 100-200 Hz), la frequenza di 40 Hz si mantiene rigorosamente costante e priva di jitter rilevante durante lo streaming. L'acquisizione dei timestamp hardware reali tramite il plugin nativo garantisce una sincronizzazione temporale precisa con i flussi visivi, requisito fondamentale per l'impiego in algoritmi di fusione sensoriale e Visual-Inertial Odometry (VIO).

4.4 Manuale d'uso

L'intero framework sviluppato per mettere in comunicazione il visore Magic Leap 2 con l'ecosistema ROS 2 è disponibile all'interno di un repository GitHub ufficiale del progetto [3]. In questa sezione viene illustrata la procedura operativa dettagliata per l'installazione, la configurazione e l'avvio del sistema, concepita in modo tale che l'utente finale non debba interagire con l'ambiente di sviluppo Unity o ricompilare il codice per utilizzare il bridge.

4.4.1 Configurazione dell'ambiente ROS 2

Sul computer host (su cui è installato ROS 2) è necessario predisporre il workspace di lavoro integrando i due package software forniti nel repository:

1. Importazione dei package: All'interno della directory `src` del proprio workspace ROS 2, clonare o copiare:
 - il package `config_ml2_stream`, deputato alla gestione e all'invio dei parametri di configurazione verso il visore;
 - il package `ros_tcp_endpoint` (versione v0.7.0 per ROS 2), responsabile della gestione del socket TCP e del routing dei messaggi di rete verso i topic ROS 2.
2. Personalizzazione del file `config.yaml`: Prima della compilazione o dell'avvio, accedere al package `config_ml2_stream` e modificare il file `config.yaml`. All'interno di questo file l'utente può abilitare o disabilitare singolarmente i vari sensori (World Cameras, telecamera RGB, Depth Camera, Eye Tracking, IMU e sensore di luminosità) e specificare parametri operativi quali framerate, modalità di stream e risoluzioni desiderate.
3. Compilazione del workspace: Eseguire la compilazione dei package tramite lo strumento `colcon` e procedere con il sourcing dell'ambiente:

```
cd ~/ros2_ws
colcon build --packages-select ros_tcp_endpoint config_ml2_stream
source install/setup.bash
```

4.4.2 Installazione e configurazione dell'applicazione su Magic Leap 2

Per eseguire lo streaming sensoriale dal visore è sufficiente installare il pacchetto precompilato (.apk) senza dover ricreare o aprire il progetto su Unity:

1. Installazione dell'APK: Collegare il visore Magic Leap 2 al PC tramite cavo USB-C e installare il file .apk (reperibile all'interno della cartella *Application* del repository) sfruttando l'applicazione ufficiale Magic Leap Hub 3.
2. Requisiti di rete: L'applicazione precompilata è configurata di fabbrica per tentare la connessione TCP verso l'indirizzo IP statico 192.168.0.203 sulla porta 10000. È pertanto fondamentale che la macchina host su cui gira ROS 2 sia configurata con tale indirizzo IP statico e che entrambi i dispositivi (PC e visore) siano connessi alla medesima sottorete locale (tramite Wi-Fi o connessione di rete via cavo USB-C).
3. Concessione dei permessi di sistema (Android Permissions): Trattandosi di un'applicazione che accede direttamente a flussi video, dati spaziali e sensori a basso livello, prima del primissimo avvio è obbligatorio concedere manualmente tutti i permessi richiesti dal sistema operativo Android (Magic Leap OS):
 - Indossare il visore e accedere al menu Settings (Impostazioni);
 - Navigare nella sezione Apps (Applicazioni) e selezionare l'applicazione del bridge;
 - Accedere alla scheda Permissions (Permessi) e abilitare esplicitamente tutte le autorizzazioni richieste (in particolare: *Camera*, *Spatial Mapping/Perception*, *Sensors* e *Local Network*). La mancata concessione di uno di questi permessi provocherebbe il blocco immediato delle routine di acquisizione a basso livello.

4.4.3 Sequenza di avvio e streaming dei dati

Per assicurare il corretto handshake di rete e consentire al visore di ricevere i parametri prima dell'inizio dello streaming, è necessario rispettare rigorosamente il seguente ordine di avvio:

1. Avvio dell'endpoint di comunicazione ROS 2: Aprire un primo terminale sull'host ed eseguire il launch file dell'endpoint TCP:

```
ros2 launch ros_tcp_endpoint endpoint.py
```

2. Avvio del server di configurazione: Aprire un secondo terminale e lanciare il nodo incaricato di leggere il file *config.yaml* e restare in ascolto della richiesta dal visore:

```
ros2 launch config_ml2_stream magic_leap_config.launch.py
```

3. Esecuzione dell'applicazione a bordo visore: Una volta verificato che i due nodi ROS 2 siano attivi e in ascolto, lanciare l'applicazione all'interno del Magic Leap 2. All'avvio, l'applicazione:

- stabilisce la connessione con l'endpoint;
- invia automaticamente il messaggio di handshake sul topic `/request`;
- riceve la configurazione JSON dal nodo ROS 2;
- sblocca le coroutine di acquisizione per i soli sensori abilitati, avviando la pubblicazione continua sui rispettivi topic ROS 2.

4.5 Modularità e upgrade

Il framework è stato ingegnerizzato seguendo un pattern modulare e disaccoppiato, consentendo a futuri sviluppatori di modificare l'applicazione a bordo visore, cambiare la configurazione di rete predefinita (es. l'indirizzo IP del server ROS 2) o integrare nuovi flussi sensoriali e funzionalità AR avanzate (come il tracciamento delle mani o algoritmi di visione a bordo).

4.5.1 Setup dell'ambiente di sviluppo Unity

Per poter modificare il codice sorgente ed effettuare una nuova compilazione del package .apk, è necessario predisporre l'ambiente di sviluppo Unity rispettando i seguenti passaggi:

1. Creazione del progetto Unity: Creare un nuovo progetto configurato con il supporto per Magic Leap 2 OpenXR, seguendo le specifiche e i requisiti indicati dalla documentazione ufficiale del costruttore [16].
2. Importazione degli script: Nella scena principale, creare un `GameObject` vuoto (che fungerà da contenitore logico dei moduli) e associarvi tutti gli script C# forniti nella cartella `Scripts` del repository GitHub [3], assicurandosi che siano tutti abilitati nell'Inspector.
3. Integrazione del plugin Android nativo:
 - Scaricare il file binario .aar presente nella cartella `Android_Plugin` del repository [3] e posizionarlo all'interno del progetto Unity nel percorso `Assets/Plugins/Android/`.
 - Per consentire a Unity di collegare correttamente il plugin scritto in Kotlin, accedere al menu *Edit > Project Settings > Player*, selezionare la scheda della piattaforma Android (icona del robottino) e, all'interno della sezione *Publishing Settings*, spuntare la casella `Custom Main Gradle Template`.

- Aprire con un editor di testo il file generato `Assets/Plugins/Android/mainTemplate.gradle`, individuare il blocco `dependencies` in fondo al file e inserire la dipendenza della libreria standard di Kotlin:

```
dependencies {
    // ... altre dipendenze generate automaticamente da Unity ...
    // Dipendenza per il plugin nativo dei sensori:
    implementation "org.jetbrains.kotlin:kotlin-stdlib:1.9.20"
}
```

4. Installazione e Configurazione del Package ROS-TCP-Connector:

- Aprire il *Package Manager* di Unity (*Window > Package Manager*), cliccare sul pulsante + in alto a sinistra e selezionare la voce *Add package from git URL*.
- Incollare il repository del connettore ROS:

```
https://github.com/Unity-Technologies/ROS-TCP-Connector.git?
path=/com.unity.robotics.ros-tcp-connector
```

- Completata l'installazione, accedere alla nuova voce di menu *Robotics* comparsa nella barra superiore di Unity: nella finestra di configurazione, selezionare *ROS2* come versione del protocollo e impostare l'indirizzo IP della propria macchina host ROS 2 qualora sia differente da quello di default (192.168.0.203).

Per ulteriori dubbi fare riferimento al repository GitHub [15].

4.5.2 Linee guida per l'estensione e l'upgrade del codice

L'architettura software consente di apportare modifiche o implementare nuovi moduli secondo uno schema logico standardizzato in tre passaggi:

1. Estensione della struttura di configurazione (`Configurator.cs`):

- Per aggiungere nuovi parametri personalizzabili via YAML (ad esempio un frame-rate custom per un nuovo sensore o flag di attivazione aggiuntivi), occorre estendere la classe interna `AppConfig` definita nello script `Configurator.cs`.
- All'avvio dell'applicazione, il metodo di parsing JSON caricherà automaticamente i nuovi valori specificati nel file `config.yaml` all'interno della struttura `AppConfig`.

2. Propagazione dei parametri ai moduli:

- Sempre all'interno dello script `Configurator.cs`, aggiungere le istruzioni necessarie per assegnare i nuovi parametri letti alle variabili pubbliche degli script dedicati prima di impostare a `true` i rispettivi flag di attivazione.

3. Sviluppo di nuovi moduli sensoriali o funzionalità AR:

- Qualsiasi nuovo modulo (come un gestore per l'Hand Tracking, algoritmi di computer vision a bordo o un renderer di ologrammi 3D per HRC) può essere implementato creando un nuovo script C# indipendente.
- Lo script dovrà semplicemente:
 - esporre una variabile booleana (es. `is_active`) controllata da `Configurator.cs`;
 - gestire nel ciclo `Update` o tramite coroutine asincrona la logica di campionamento e serializzazione verso i publisher del connettore ROS 2;
 - appoggiarsi eventualmente al plugin Android nativo qualora fosse necessario estrarre ulteriori metadati hardware di basso livello dal sistema operativo.

Capitolo 5

Validazione dei risultati sperimentali

5.1 Setup sperimentale e procedura di acquisizione

Al fine di validare l'accuratezza temporale, l'integrità dei flussi video e la stabilità complessiva del bridge sviluppato, il framework è stato testato integrando l'architettura con un algoritmo di localizzazione e mappatura visiva (Visual SLAM). Nello specifico, per la stima della traiettoria è stato impiegato ORB-SLAM3 in modalità monoculare (Mono SLAM), alimentato dai flussi delle telecamere di tracciamento ambientale (World Cameras). L'attività sperimentale è stata condotta all'interno di un ambiente indoor non strutturato. Per isolare il comportamento di ciascun sensore ottico e garantire l'assenza di interferenze o colli di bottiglia legati alla saturazione di banda, i test sono stati eseguiti abilitando un solo flusso alla volta:

1. Configurazione del sensore: Tramite il file `config.yaml`, è stata attivata singolarmente la prima telecamera (World Center).
2. Avvio e streaming: Sono stati inizializzati l'endpoint ROS 2, il nodo di configurazione e l'applicazione a bordo del Magic Leap 2, avviando lo streaming a 30 Hz del relativo topic `/magic_leap2/world_center/image_raw` e dei messaggi di `CameraInfo`.
3. Esecuzione dello SLAM e registrazione dati: È stato avviato il nodo di ORB-SLAM3 monoculare e, contestualmente, è stata avviata la registrazione su `ros2 bag` contenente:
 - le pose stimate in tempo reale dall'algoritmo di SLAM;
 - le pose stimate contemporaneamente dal sistema di localizzazione interno del Magic Leap 2 (pubblicate sul topic `/tf`), utilizzate come traiettoria di riferimento (pseudo ground truth).

4. Iterazione sui moduli rimanenti: La sessione di test è stata terminata e ripetuta in modo analogo per le restanti due telecamere (World Left e World Right), eseguendo per ciascuna un percorso di esplorazione indoor diverso.

Le pose fornite dal sistema interno del Magic Leap 2 sono state scelte come ground truth per la loro elevata affidabilità. Sfruttando la fusione tra più telecamere, sensore di profondità e IMU per ancorare gli ologrammi nello spazio, il visore garantisce una precisione nettamente superiore rispetto a ORB-SLAM3 in configurazione monoculare.

5.2 Metodologia di analisi e segmentazione con evo

La valutazione quantitativa dell'errore di stima è stata effettuata mediante il pacchetto open-source evo [17] (Python package for the evaluation of odometry and SLAM), standard di riferimento nella comunità robotica per il confronto e il benchmark di traiettorie 3D.

5.2.1 Motivazione della segmentazione dei dati

L'utilizzo del Visual SLAM in modalità puramente monoculare in ambienti indoor presenta limiti intrinseci noti in letteratura: la mancanza di informazione diretta sulla profondità metrica e la presenza di aree a basso gradiente di texture possono causare temporanee perdite del tracciamento delle feature (tracking loss), inducendo derive locali o reset dell'algoritmo. Poiché l'obiettivo primario del test è verificare la fedeltà del framework (ovvero accertare che i frame video non subiscano frame drop, jitter anomalo o disallineamenti nei timestamp tali da corrompere la stima del moto), l'analisi non ha preso in esame le traiettorie complessive qualora interrotte da perdite di tracking algoritmiche. Dalle registrazioni rosbag sono stati estratti e analizzati segmenti temporali continui e privi di salti, all'interno dei quali il tracciamento monoculare si è mantenuto ininterrotto.

5.2.2 Metriche di errore e allineamento traiettorie

L'analisi quantitativa è stata condotta confrontando la traiettoria stimata da ORB-SLAM3 ($\mathbf{P}_{\text{est}}$) con la traiettoria di riferimento fornita dal tracking on-board del visore ($\mathbf{P}_{\text{ref}}$).

Allineamento spaziale e temporale ($\text{Sim}(3)$ Umeyama)

Nel Mono SLAM, l'assenza di sensori di profondità o vincoli inerziali diretti rende la scala metrica non direttamente osservabile. Di conseguenza, la traiettoria stimata è definita a meno di un fattore di scala arbitrario $s > 0$ e di una rototraslazione iniziale rispetto al sistema di coordinate

di riferimento. Per consentire un confronto coerente, le pose sono state associate temporalmente in base ai timestamp e allineate nello spazio mediante l'algoritmo di Umeyama [18], minimizzando l'errore quadratico medio per trovare la trasformazione di similarità $\mathbf{S} \in \text{Sim}(3)$ ottimale composta da:

- una rotazione $\mathbf{R} \in \text{SO}(3)$;
- una traslazione $\mathbf{t} \in \mathbb{R}^3$;
- un fattore di scala $s \in \mathbb{R}^+$.

La posa stimata allineata all'istante i -esimo è quindi espressa come:

$$\mathbf{p}_{\text{est},i}^{\text{aligned}} = s\mathbf{R}\mathbf{p}_{\text{est},i} + \mathbf{t}$$

Absolute Pose Error (APE)

Come metrica primaria di accuratezza globale è stato calcolato l'Absolute Pose Error (APE) applicato alla componente traslazionale. Per ciascuna coppia di pose corrispondenti all'istante i , l'errore puntuale è definito come la distanza euclidea:

$$e_i = \|\mathbf{p}_{\text{est},i}^{\text{aligned}} - \mathbf{p}_{\text{ref},i}\|$$

Sull'insieme degli N campioni del segmento temporale, sono stati estratti i seguenti indici statistici:

- Root Mean Square Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}$$

- Errore Medio (μ) e Deviazione Standard (σ):

$$\mu = \frac{1}{N} \sum_{i=1}^N e_i, \quad \sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (e_i - \mu)^2}$$

- Errore Mediano, Minimo e Massimo.

5.2.3 Decomposizione di traslazione e orientamento

Parallelamente all'APE scalare, sono state analizzate:

- Le componenti cartesiane individuali $(x(t), y(t), z(t))$ per verificare la presenza di eventuali derivate lungo specifici assi (es. l'asse z verticale).
- L'orientamento spaziale espresso in angoli di Eulero (roll, pitch, yaw), per confermare la stabilità dell'inseguimento rotazionale e l'assenza di sfasamenti temporali nei dati inviati dal bridge.

5.3 Risultati sperimentali e analisi delle traiettorie

5.3.1 Validazione su World Center Camera — Traiettoria 1

La prima sessione di validazione ha previsto l’acquisizione di una traiettoria indoor continua della durata di circa 13.5 s, per uno sviluppo complessivo del percorso stimato pari a 5.304 m (5.359 m registrati dal sistema di riferimento on-board). Durante l’esecuzione, il bridge ha trasmesso il flusso video monocromatico a piena risoluzione (1016×1016 pixel a 30 Hz) e le pose native verso l’host ROS 2. Dall’allineamento temporale sono state estratte ed associate univocamente 367 coppie di pose con uno scarto temporale massimo inferiore a 0.01 s ($\Delta t_{\max} \leq 10$ ms), confermando l’assenza di ritardi sistematici o deriva nei timestamp del middleware.

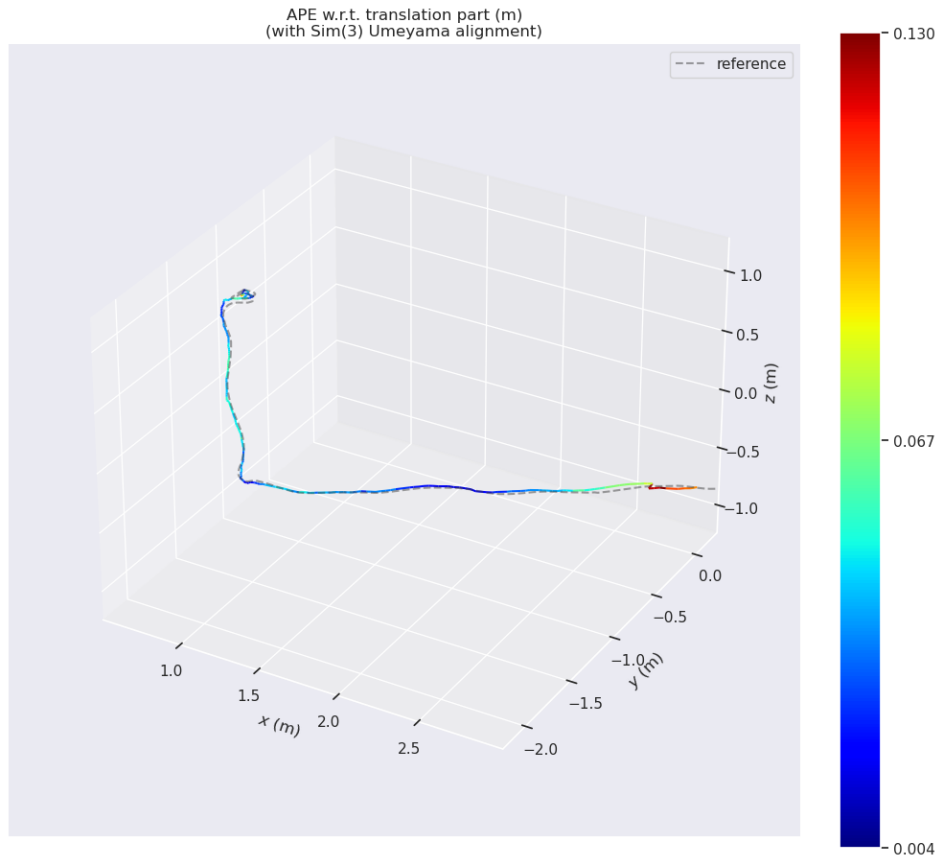


Figura 5.1: Ricostruzione 3D della traiettoria stimata e confronto con la ground truth, con mappa cromatica dell’errore APE di traslazione dopo allineamento Sim(3) (Umeyama).

Metrica	RMSE [m]	Media (μ) [m]	Mediana [m]	Std (σ) [m]	Min [m]	Max [m]
APE (Traslazione)	0.0440	0.0389	0.0389	0.0206	0.0036	0.1297

Tabella 5.1: Metriche statistiche di errore APE (traslazione) ottenute con allineamento Sim(3) Umeyama per la Traiettoria 1 (World Center).

Analisi del profilo di errore assoluto della posa

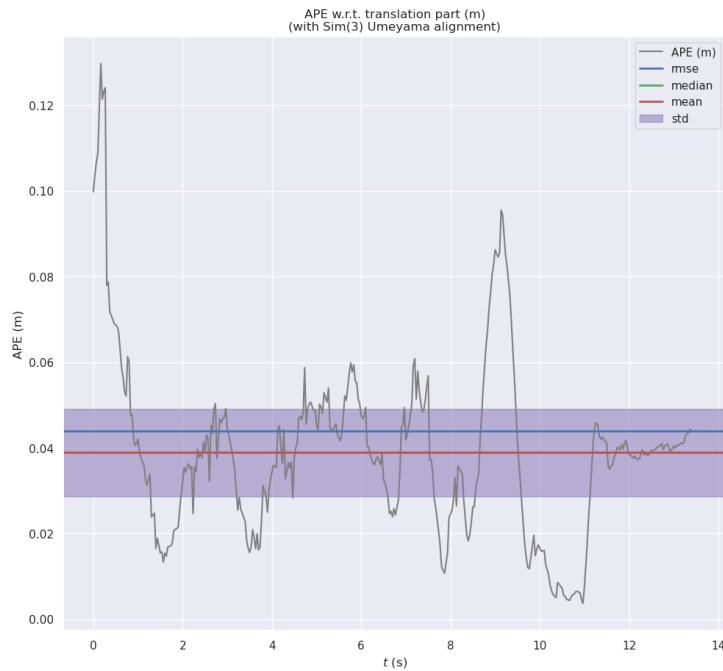


Figura 5.2: Andamento temporale dell’Absolute Pose Error (APE) di traslazione con indicazione di RMSE, media, mediana e deviazione standard ($\pm\sigma$).

L’errore quadratico medio ottenuto sulla componente traslazionale si attesta a $RMSE = 4.40$ cm, con un valore medio di $\mu = 3.89$ cm e una dispersione contenuta ($\sigma = 2.06$ cm). Come evidenziato dal profilo temporale dell’APE:

- L’errore si mantiene per la quasi totalità dell’acquisizione al di sotto dei 5 cm, con valori minimi nell’ordine dei 3.6 mm durante i tratti a traslazione rettilinea uniforme.
- Il picco massimo (≈ 12.97 cm) si colloca nei transitori iniziali di inizializzazione della mappa monoculare (primi istanti di convergenza del Bundle Adjustment locale) e in corrispondenza del cambio repentino di direzione attorno a $t \approx 9$ s.

Allineamento della traiettoria e decomposizione degli assi (X, Y, Z)

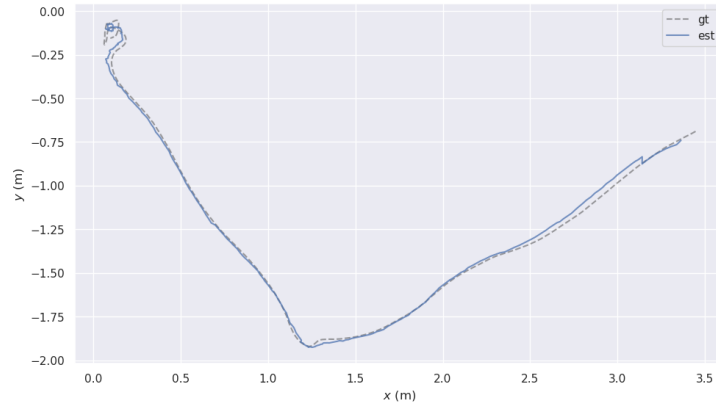


Figura 5.3: Proiezione bidimensionale sul piano XY della traiettoria stimata a confronto con la traiettoria di riferimento.

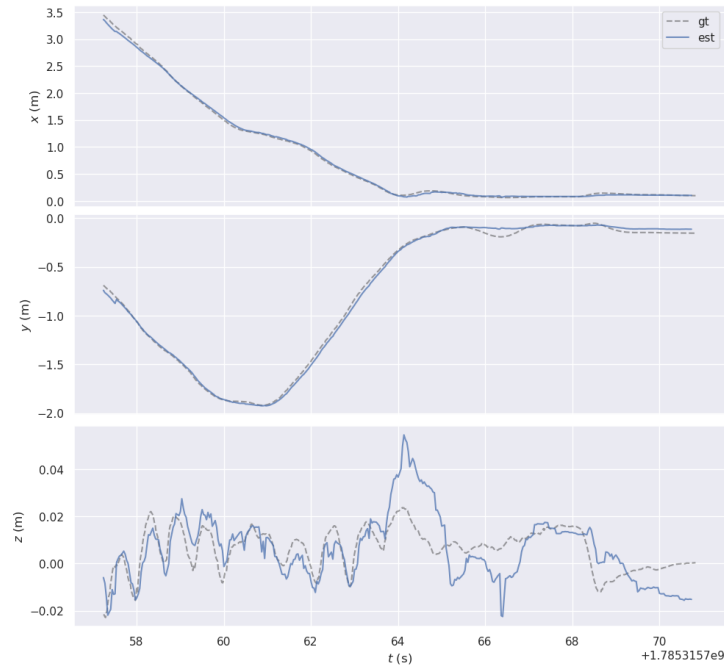


Figura 5.4: Evoluzione temporale delle singole componenti cartesiane di posizione ($x(t)$, $y(t)$, $z(t)$): confronto tra stima e ground truth.

Il confronto planare sul piano XY e nello spazio 3D evidenzia una fedeltà geometrica elevata rispetto alla ground-truth del visore. La scomposizione cartesiana nel dominio del tempo evidenzia che:

- Gli assi orizzontali $x(t)$ e $y(t)$ seguono fedelmente la cinematica reale, preservando cambi di curvatura e decelerazioni.

- L'asse verticale $z(t)$ mostra oscillazioni minime confinate in un range di ± 2 cm, tipico delle lievi oscillazioni della camminata dell'operatore.

Stima dell'orientamento (roll, pitch, yaw)

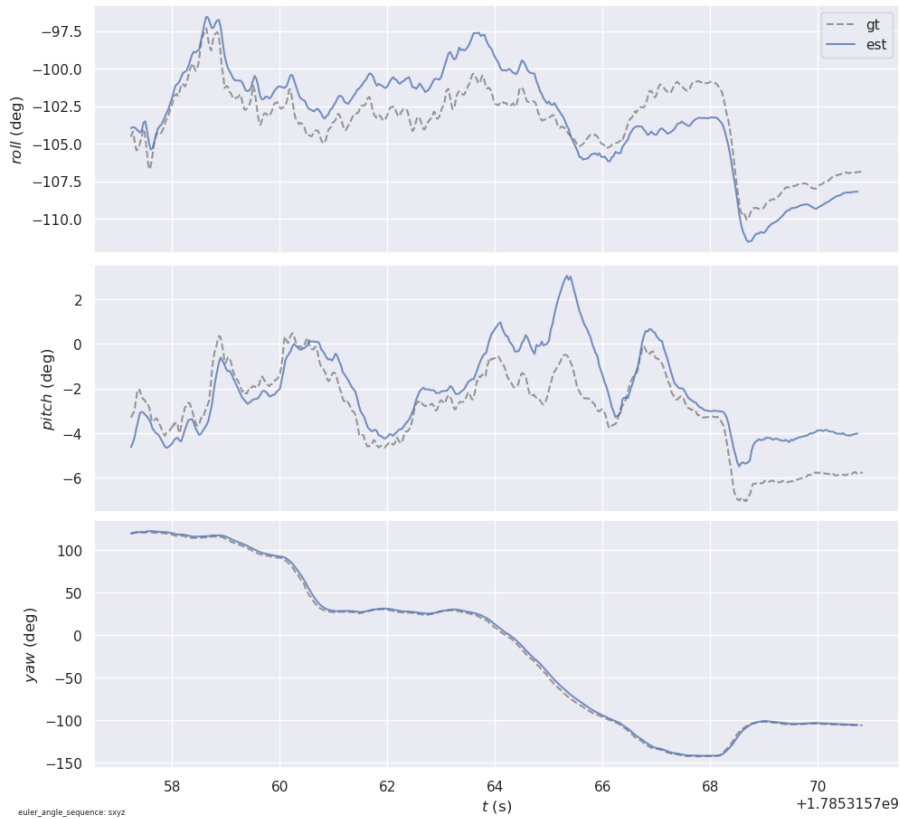


Figura 5.5: Confronto temporale degli angoli di Eulero (Roll, Pitch, Yaw) stimati rispetto al riferimento nativo on-board del visore.

L'analisi degli angoli di Eulero conferma la coerenza dinamica dei frame trasmessi dal bridge:

- L'angolo di imbardata (yaw), che compie un'ampia rotazione di oltre 200° durante l'esplorazione (da $+120^\circ$ a -105°), viene tracciato con sovrapposizione pressoché perfetta rispetto al riferimento nativo.
- Gli angoli di roll e pitch presentano scostamenti angolari medi inferiori a $1.5^\circ - 2^\circ$, limitati unicamente alle naturali differenze tra la stima puramente visuale di ORB-SLAM3 e la fusione visivo-inerziale a bordo del Magic Leap 2.

5.3.2 Validazione su World Center Camera — Traiettorie 2

A conferma della ripetibilità dei risultati, è stata eseguita una seconda sessione di validazione su un percorso indoor differente, caratterizzato da una traiettoria continua della durata di circa 10.9 s, per uno sviluppo complessivo stimato pari a 6.311 m (5.960 m registrati dal sistema di riferimento on-board). Anche in questa acquisizione, il bridge ha trasmesso il flusso video monocromatico a 1016×1016 pixel a 30 Hz unitamente alle pose native del visore. Dalla sincronizzazione temporale sono state associate univocamente 305 coppie di pose con uno scarto temporale massimo inferiore a 0.01 s ($\Delta t_{\max} \leq 10$ ms), confermando nuovamente la stabilità della propagazione dei timestamp.

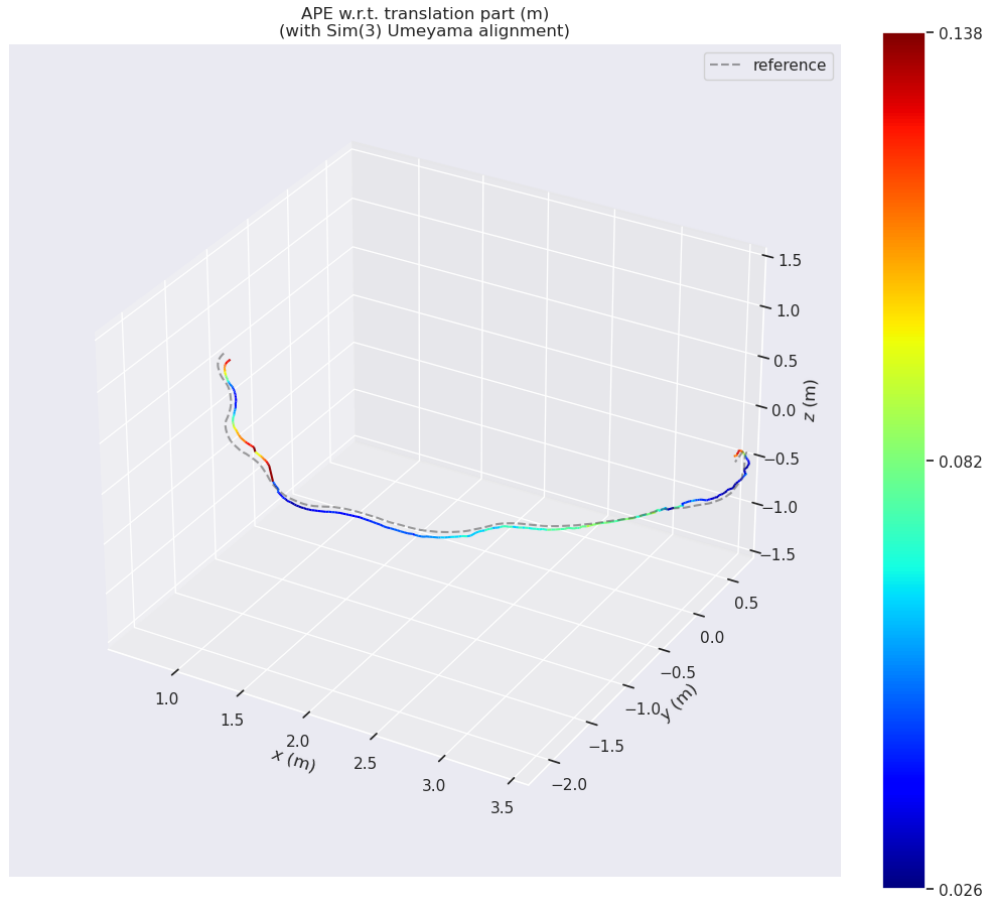


Figura 5.6: Ricostruzione 3D della traiettoria stimata e confronto con la ground truth, con mappa cromatica dell'errore APE di traslazione dopo allineamento Sim(3) (Umeyama).

Metrica	RMSE [m]	Media (μ) [m]	Mediana [m]	Std (σ) [m]	Min [m]	Max [m]
APE (Traslazione)	0.0746	0.0681	0.0621	0.0305	0.0261	0.1378

Tabella 5.2: Metriche statistiche di errore APE (traslazione) ottenute con allineamento Sim(3) Umeyama per la Traiettorie 2 (World Center).

Analisi del profilo di errore assoluto (APE)

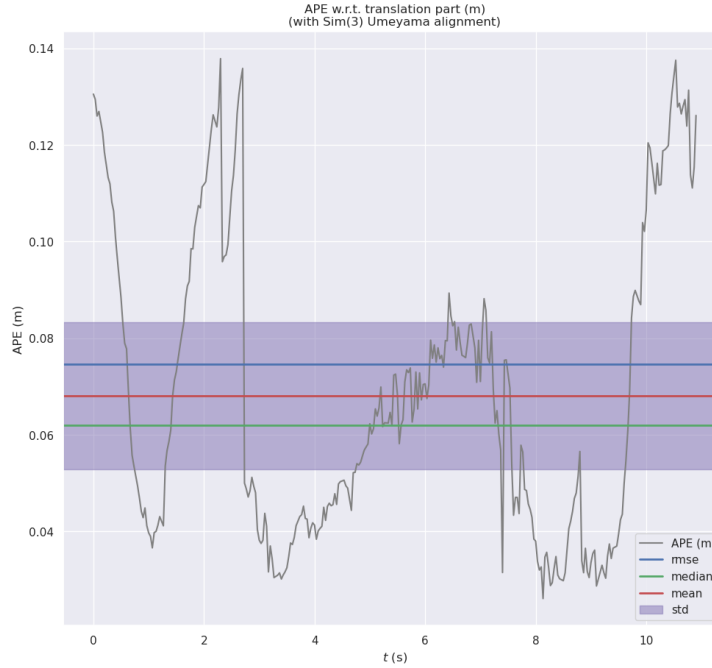


Figura 5.7: Andamento temporale dell’Absolute Pose Error (APE) di traslazione con indicazione di RMSE, media, mediana e deviazione standard ($\pm\sigma$).

L’errore quadratico medio ottenuto sulla componente traslazionale si attesta a $RMSE = 7.46$ cm, con un valore medio di $\mu = 6.81$ cm, una mediana di 6.21 cm e una dispersione statistica pari a $\sigma = 3.05$ cm. Come evidenziato dal profilo temporale dell’APE: I picchi di errore più marcati (≈ 13.78 cm) si concentrano nella fase iniziale di convergenza dell’algoritmo ($t \approx 0 - 0.5$ s) e in corrispondenza del cambio repentino di direzione attorno a $t \approx 9 - 10$ s, dove la dinamica angolare rapida introduce una lieve discrepanza temporanea nella stima monoculare.

- L’errore si mantiene stabilmente al di sotto degli 8 cm per l’intera durata della corsa centrale, attestandosi su valori minimi di 2.61 cm.
- I picchi di errore più marcati (≈ 13.78 cm) si concentrano nella fase iniziale di convergenza dell’algoritmo ($t \approx 0 - 0.5$ s) e in corrispondenza del cambio repentino di direzione attorno a $t \approx 9 - 10$ s, dove la dinamica angolare rapida introduce una lieve discrepanza temporanea nella stima monoculare.

allineamento della traiettoria e decomposizione degli assi (X, Y, Z)

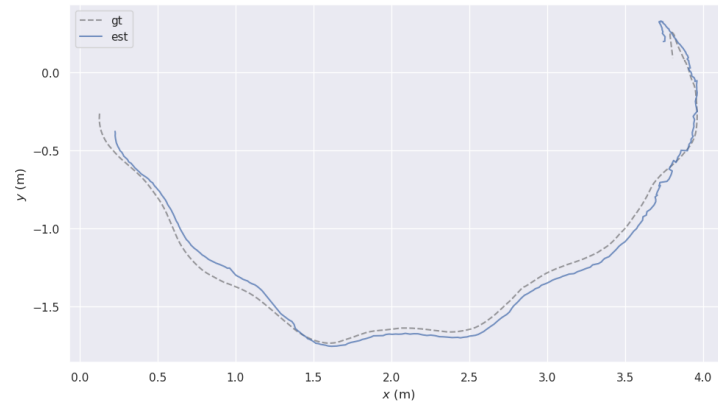


Figura 5.8: Proiezione bidimensionale sul piano XY della traiettoria stimata a confronto con la traiettoria di riferimento.

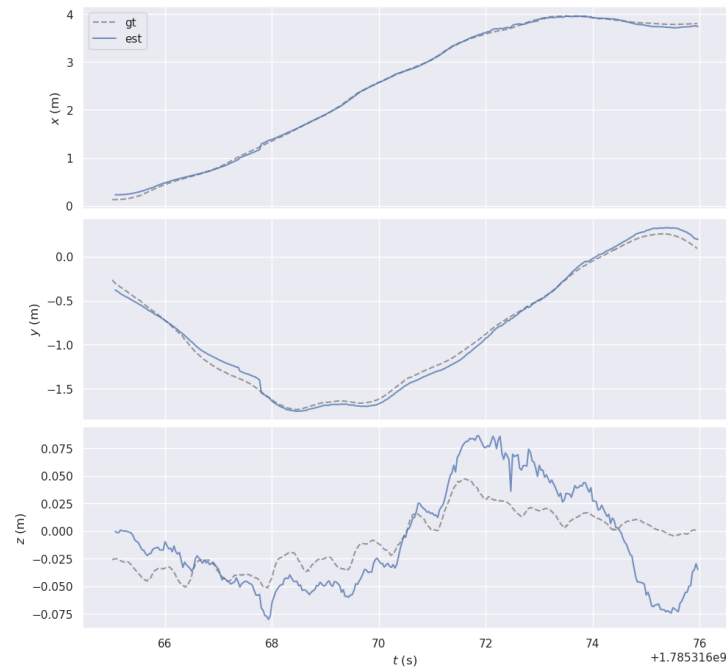


Figura 5.9: Evoluzione temporale delle singole componenti cartesiane di posizione ($x(t)$, $y(t)$, $z(t)$): confronto tra stima e ground truth.

Il confronto spaziale sul piano XY e nello spazio tridimensionale mostra un'aderenza geometrica notevole lungo l'intero sviluppo curvilineo:

- Gli assi orizzontali $x(t)$ e $y(t)$ riproducono fedelmente l'evoluzione temporale della posizione, seguendo con continuità sia le fasi rettilinee sia i tratti a curvatura variabile.

- L'asse verticale $z(t)$ manifesta oscillazioni contenute nell'ordine di pochi centimetri, senza evidenziare derive verticali divergenti o perdita di consistenza metrica nel tempo.

Stima dell'orientamento (roll, pitch, yaw)

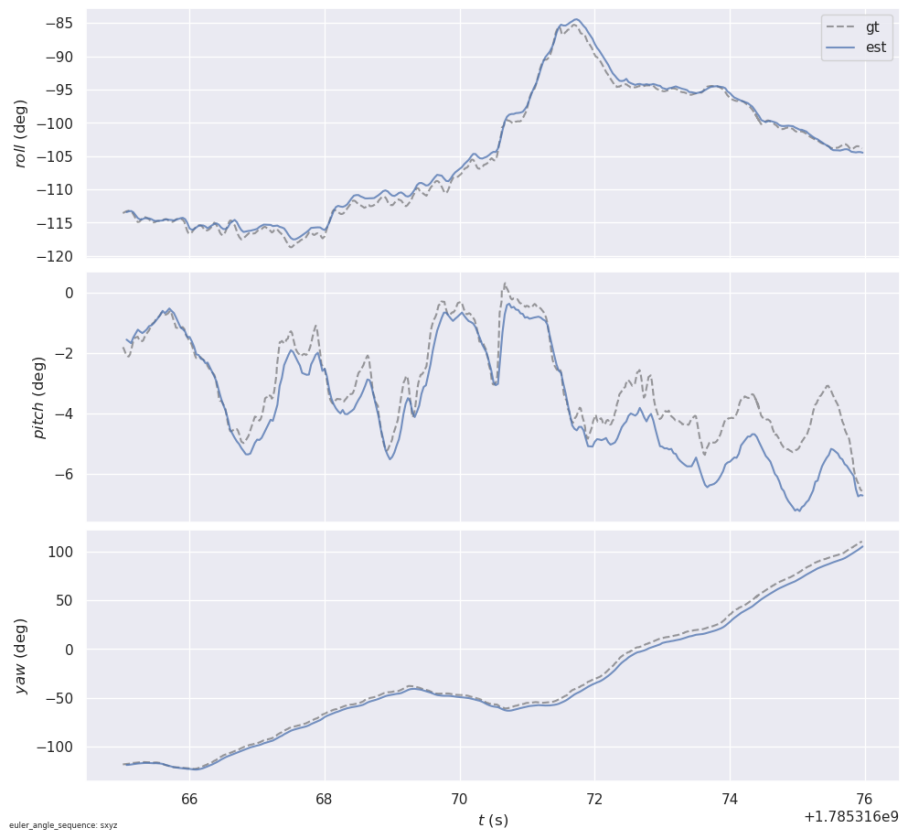


Figura 5.10: Confronto temporale degli angoli di Eulero (Roll, Pitch, Yaw) stimati rispetto al riferimento nativo on-board del visore.

L'analisi dell'orientamento spaziale espresso in angoli di Eulero conferma l'assenza di sfasamenti o ritardi di propagazione tra i frame:

- L'angolo di imbardata (yaw), che sperimenta un'ampia rotazione progressiva di oltre 220° (da -120° fino a oltre $+105^\circ$), viene tracciato con eccellente precisione e sovrapposizione continua rispetto alla ground truth del visore.
- Gli angoli di rollio (roll) e beccheggio (pitch) seguono fedelmente il profilo dinamico registrato dal Magic Leap 2, presentando scostamenti minimi interamente imputabili alla diversa natura dei due estimatori (ottico monoculare puro contro fusione visivo-inerziale hardware).

5.3.3 Validazione su World Left Camera — Traiettoria 3

La terza sessione sperimentale ha avuto l'obiettivo di testare il modulo sensoriale della telecamera laterale sinistra (World Left) su un percorso esteso e a circuito chiuso (loop), in cui l'operatore compie una traiettoria completa ritornando in prossimità della posizione iniziale. L'acquisizione ha registrato una durata di circa 22.77 s, con uno sviluppo complessivo stimato pari a 11.961 m a fronte dei 12.277 m rilevati dal sistema di localizzazione on-board del visore. Dalla sincronizzazione temporale sono state estratte ed associate univocamente 612 coppie di pose con uno scarto temporale massimo $\Delta t_{\max} \leq 10$ ms, confermando la regolarità di ricezione dei frame anche durante sessioni di scansione prolungate.

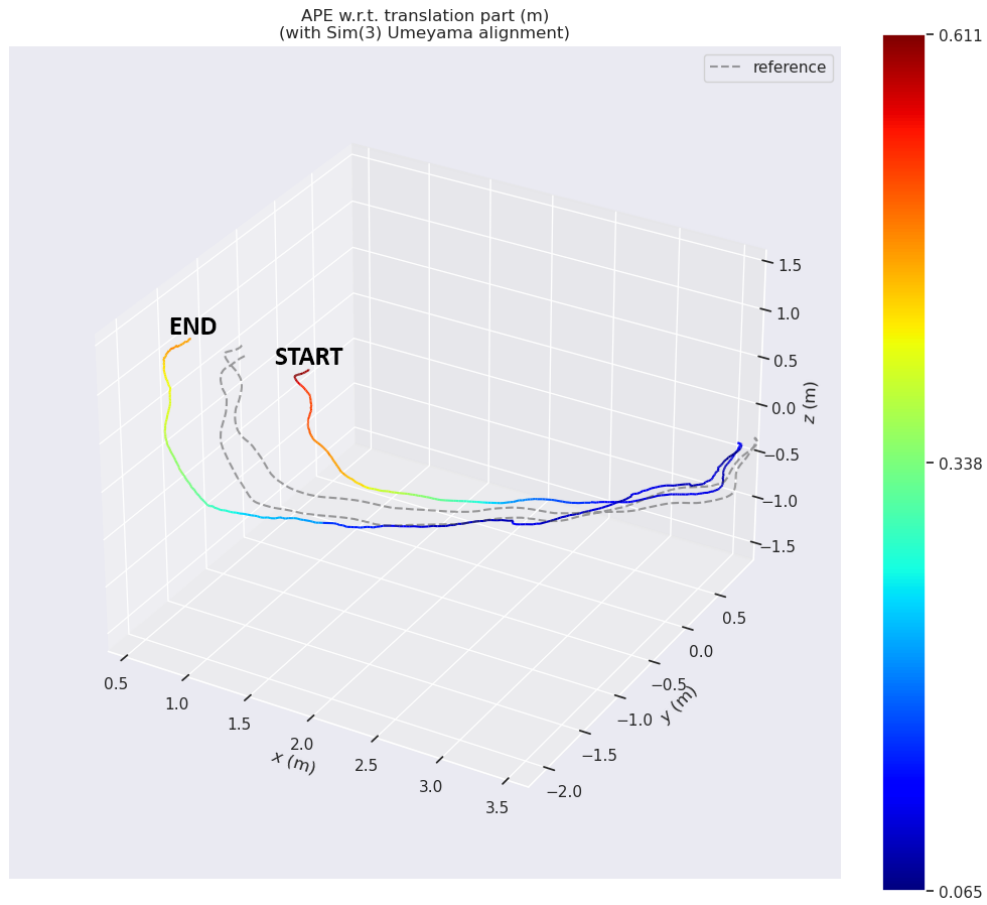


Figura 5.11: Ricostruzione 3D della traiettoria stimata e confronto con la ground truth, con mappa cromatica dell'errore APE di traslazione dopo allineamento Sim(3) (Umeyama).

Metrica	RMSE [m]	Media (μ) [m]	Mediana [m]	Std (σ) [m]	Min [m]	Max [m]
APE (Traslazione)	0.3259	0.2765	0.2314	0.1724	0.0647	0.6109

Tabella 5.3: Metriche statistiche di errore APE (traslazione) ottenute con allineamento Sim(3) Umeyama per la Traiettoria 3 (World Left).

Analisi del profilo di errore assoluto (APE)

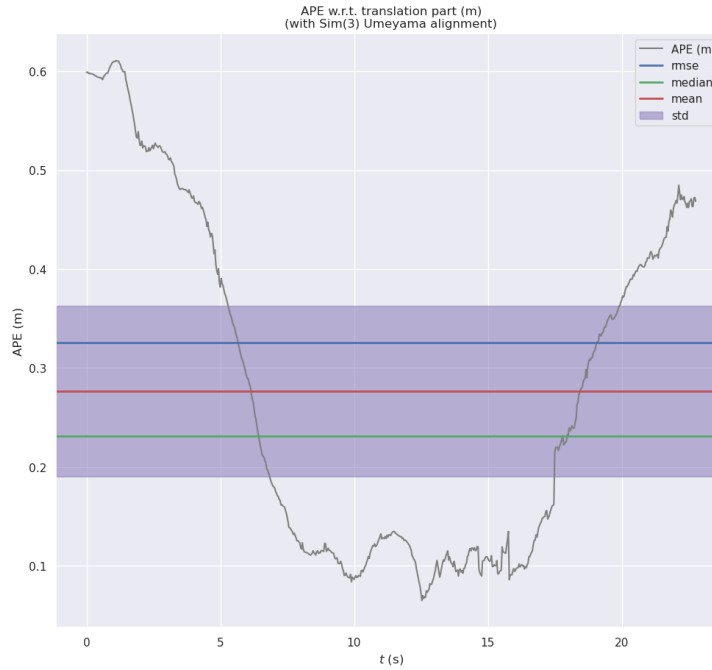


Figura 5.12: Andamento temporale dell’Absolute Pose Error (APE) di traslazione con indicazione di RMSE, media, mediana e deviazione standard ($\pm\sigma$).

L’errore quadratico medio sulla componente traslazionale si attesta a $\text{RMSE} = 32.59 \text{ cm}$, con un valore medio di $\mu = 27.65 \text{ cm}$, una mediana di 23.14 cm e una dispersione statistica $\sigma = 17.24 \text{ cm}$. Trattandosi del percorso più lungo e cinematicamente impegnativo tra quelli testati, l’aumento dell’errore assoluto rispetto alle acquisizioni precedenti riflette la naturale accumulazione di deriva (drift) e lievi incertezze di scala tipiche del Mono SLAM in assenza di vincoli metrici diretti. Come evidenziato dal profilo temporale dell’APE:

- L’errore scende sensibilmente nella fase centrale del percorso ($t \approx 8\text{--}17 \text{ s}$), mantenendosi costantemente al di sotto dei 15 cm e toccando un minimo di 6.47 cm lungo i segmenti a traslazione regolare.
- I valori più elevati ($\approx 61.09 \text{ cm}$) si registrano nelle fasi di transitorio iniziale e nella porzione finale del rientro, dove la combinazione di una lunga catena odometrica e cambi di prospettiva accentua lo scostamento tra la stima visiva pura, ottenuta con ORB-SLAM3, e la fusione inerziale del visore utilizzata come pseudo ground-truth.

Allineamento della traiettoria e decomposizione degli assi (X, Y, Z)

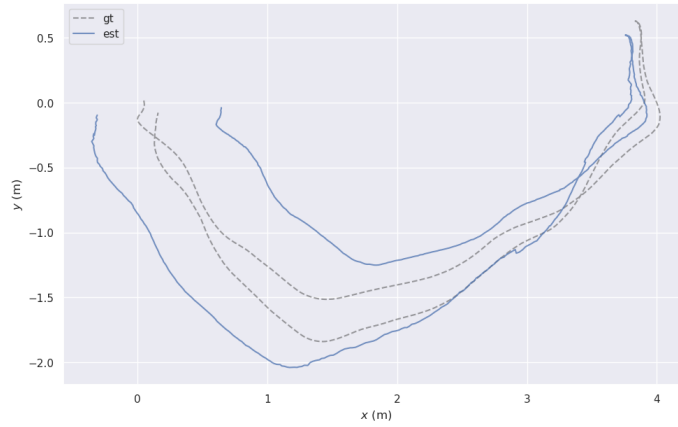


Figura 5.13: Proiezione bidimensionale sul piano XY della traiettoria stimata a confronto con la traiettoria di riferimento.

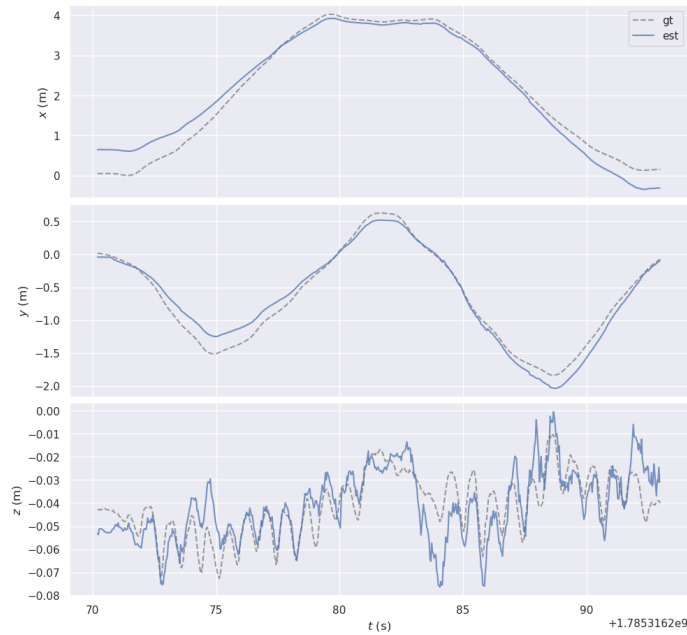


Figura 5.14: Evoluzione temporale delle singole componenti cartesiane di posizione ($x(t)$, $y(t)$, $z(t)$): confronto tra stima e ground truth.

Il confronto sul piano XY e nello spazio tridimensionale evidenzia una solida coerenza topologica: la traiettoria stimata ricalca fedelmente l'anello di andata e ritorno percorso dall'operatore. L'analisi disaccoppiata lungo i singoli assi cartesiani mostra che:

- Le componenti planari $x(t)$ (escursione da 0 m a circa 4.0 m e ritorno) e $y(t)$ (escursione fino a -2.0 m) riproducono l'andamento del moto reale con ottima correlazione di fase, confermando l'integrità del flusso temporale trasmesso dal middleware.

- L'asse verticale $z(t)$ si mantiene confinato in una banda di oscillazione contenuta (tra -0.08 m e 0.00 m), evidenziando il passo della camminata senza divergenze verticali o collassi strutturali della mappa.

Stima dell'orientamento (roll, pitch, yaw)



Figura 5.15: Confronto temporale degli angoli di Eulero (Roll, Pitch, Yaw) stimati rispetto al riferimento nativo on-board del visore.

L'analisi dell'orientamento spaziale espresso in angoli di Eulero conferma l'elevata affidabilità della componente rotazionale:

- L'angolo di imbardata (yaw) segue con eccellente sovrapposizione una rotazione complessa ed estesa di oltre 250° (con escursione da -80° fino a oltre $+170^\circ$, per poi riallinearsi a -20°), dimostrando la perfetta sincronizzazione e l'assenza di perdita di frame durante manovre di rotazione rapida.
- Gli angoli di rollio (roll) e beccheggio (pitch) tracciano con precisione le dinamiche ad alta frequenza indotte dal movimento della testa dell'operatore, confermando la totale stabilità e la fedeltà dello streaming dati anche su ottiche laterali.

5.3.4 Validazione su World Right Camera — Traiettorie 4

A completamento della campagna sperimentale sui sensori di tracciamento ottico, la quarta sessione di validazione ha previsto il test del modulo laterale destro (World Right). L'acquisizione ha riguardato un percorso indoor continuo della durata di circa 15.97 s, per uno sviluppo complessivo stimato pari a 9.206 m (a fronte dei 7.966 m registrati dal sistema di tracciamento nativo on-board). Il bridge ha trasmesso stabilmente il flusso video monocromatico a pieno frame rate unitamente alle pose del visore. Dalla sincronizzazione temporale sono state estratte ed associate univocamente 438 coppie di pose con uno scarto temporale massimo $\Delta t_{\max} \leq 10$ ms, riaffermando la perfetta sincronia e l'assenza di latenze cumulative introdotte dal middleware.

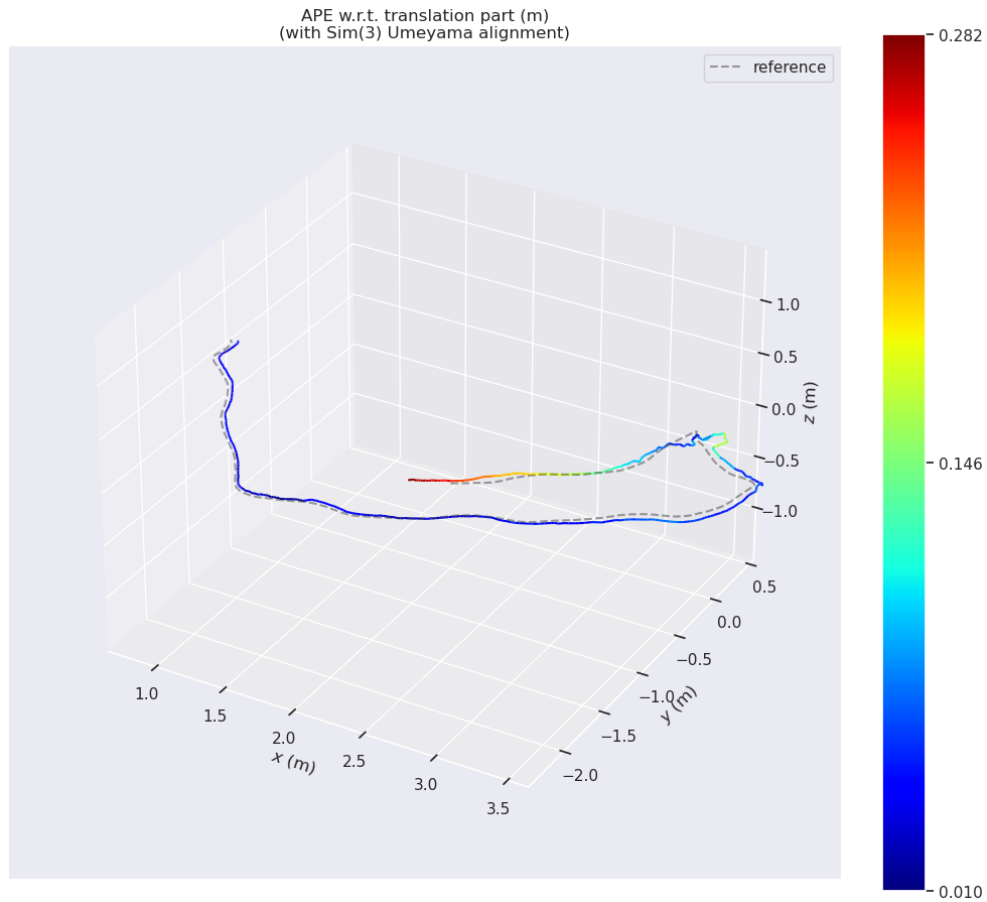


Figura 5.16: Ricostruzione 3D della traiettoria stimata e confronto con la ground truth, con mappa cromatica dell'errore APE di traslazione dopo allineamento Sim(3) (Umeyama).

Metrica	RMSE [m]	Media (μ) [m]	Mediana [m]	Std (σ) [m]	Min [m]	Max [m]
APE (Traslazione)	0.0966	0.0746	0.0475	0.0614	0.0105	0.2819

Tabella 5.4: Metriche statistiche di errore APE (traslazione) ottenute con allineamento Sim(3) Umeyama per la Traiettorie 4 (World Right).

Analisi del profilo di errore assoluto (APE)

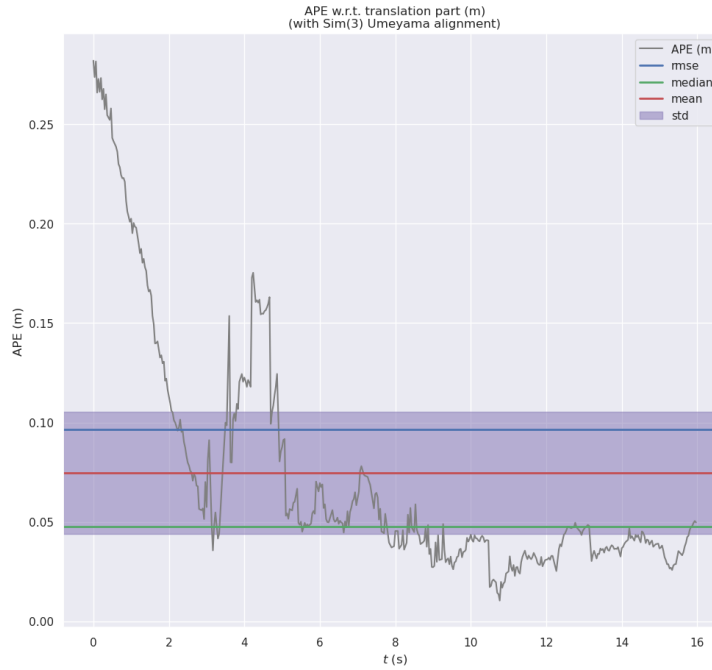


Figura 5.17: Andamento temporale dell’Absolute Pose Error (APE) di traslazione con indicazione di RMSE, media, mediana e deviazione standard ($\pm\sigma$).

L’errore quadratico medio ottenuto sulla componente traslazionale si attesta a $\text{RMSE} = 9.66 \text{ cm}$, con un valore medio di $\mu = 7.46 \text{ cm}$, una mediana pari a soli 4.75 cm e una dispersione statistica contenuta ($\sigma = 6.14 \text{ cm}$). Il profilo temporale dell’APE evidenzia l’ottima stabilità globale dell’inseguimento:

- Superata la fase transitoria iniziale, l’errore cala rapidamente e si stabilizza per la quasi totalità della sessione ($t > 8 \text{ s}$) attorno a valori inferiori ai 5 cm , con minimi nell’ordine di 1.05 cm lungo le fasi a velocità uniforme.
- Il picco massimo ($\approx 28.19 \text{ cm}$) si concentra nei primi istanti di inizializzazione ($t \approx 0 - 2 \text{ s}$) per poi riassorbirsi prontamente grazie all’ottimizzazione locale del *Bundle Adjustment*.

Allineamento della traiettoria e decomposizione degli assi (X, Y, Z)

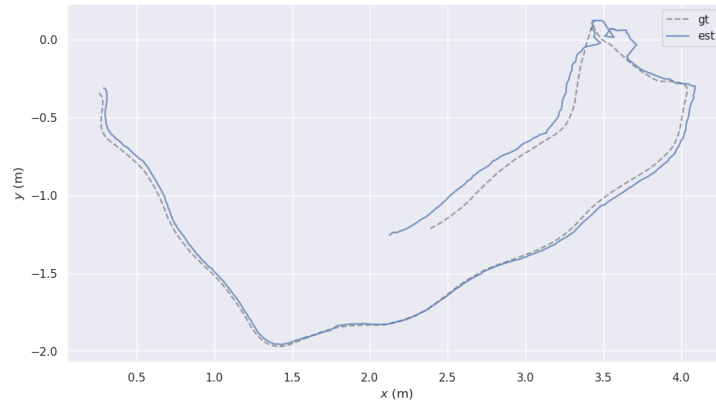


Figura 5.18: Proiezione bidimensionale sul piano XY della traiettoria stimata a confronto con la traiettoria di riferimento.

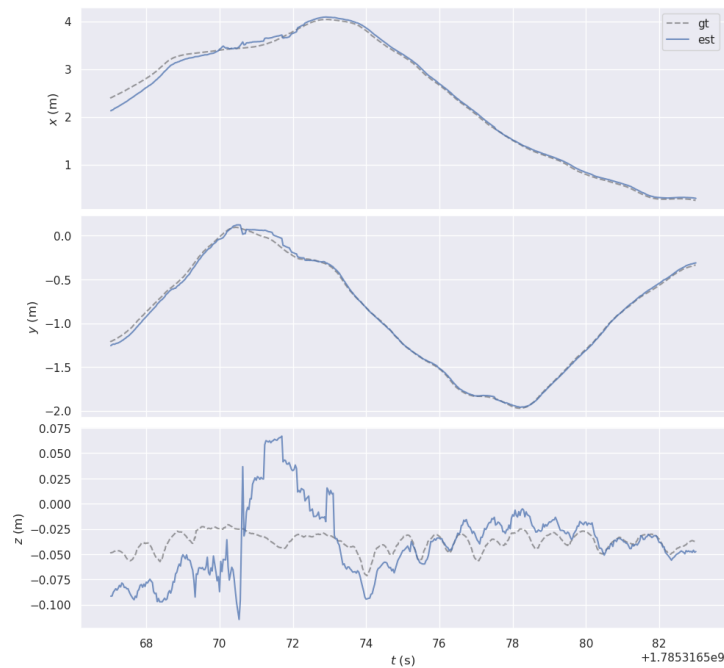


Figura 5.19: Evoluzione temporale delle singole componenti cartesiane di posizione ($x(t)$, $y(t)$, $z(t)$): confronto tra stima e ground truth.

La proiezione planare sul piano XY e nello spazio tridimensionale mostra una notevole conformità geometrica con il riferimento ground truth. La scomposizione cartesiana nel dominio del tempo evidenzia che:

- Gli assi orizzontali $x(t)$ (con escursione da 0.5 m a circa 4.0 m) e $y(t)$ (da 0.1 m a circa -2.0 m) seguono fedelmente la cinematica reale, preservando cambi di direzione e curve ad ampio raggio con eccellente correlazione di fase.

- L'asse verticale $z(t)$, pur mostrando una lieve oscillazione locale nella fase centrale ($t \approx 7.0 - 7.4$ s) dovuta a repentini aggiustamenti di scala locali, rientra prontamente a regime mantenendo scostamenti confinati in un range di pochi centimetri, senza evidenziare derivate verticali divergenti.

Stima dell'orientamento (roll, pitch, yaw)

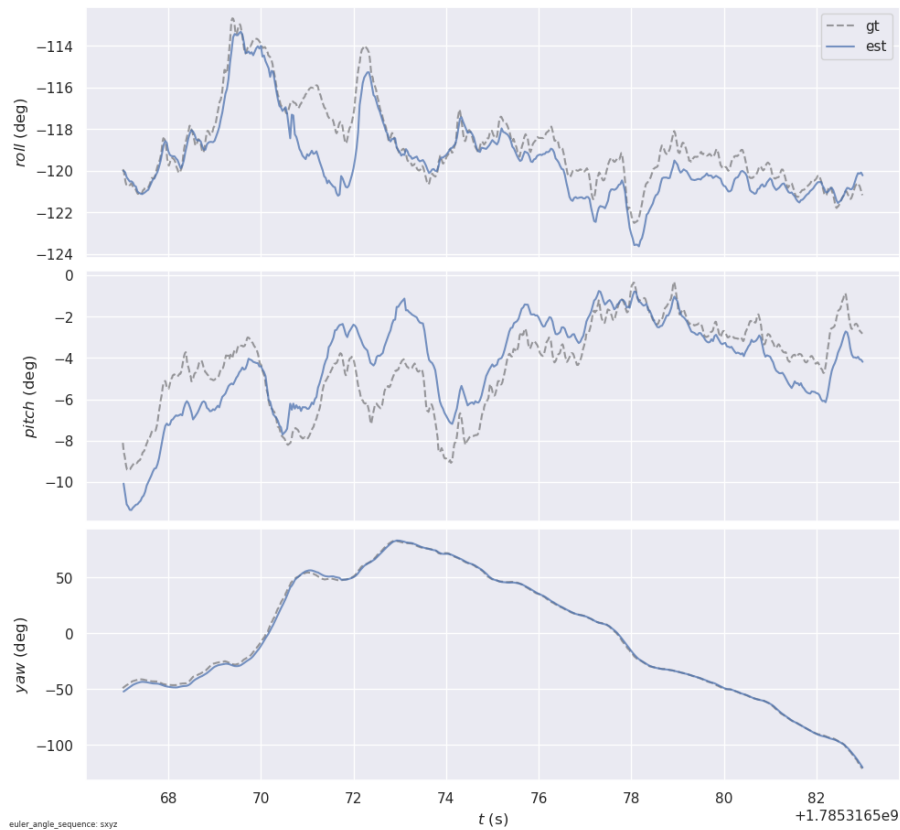


Figura 5.20: Confronto temporale degli angoli di Eulero (Roll, Pitch, Yaw) stimati rispetto al riferimento nativo on-board del visore.

L'analisi dell'orientamento spaziale espresso in angoli di Eulero conferma l'elevata precisione cinematica dei frame inviati:

- L'angolo di imbardata (yaw) segue con eccezionale sovrapposizione l'intera rotazione continua di circa 200° (con escursione da -50° fino a oltre $+80^\circ$ per poi scendere a -120°), confermando l'assenza di ritardi o salti nei timestamp del flusso video.
- Gli angoli di rollio (roll) e beccheggio (pitch) riproducono fedelmente le oscillazioni dinamiche della testa dell'operatore, mantenendo scostamenti angolari medi contenuti entro $1.5^\circ - 2.5^\circ$, imputabili unicamente alla differenza metodologica tra lo stimatore puramente visivo e la fusione inerziale proprietaria del visore.

5.3.5 Considerazioni finali sui risultati

La fase di sperimentazione ha permesso di valutare l'efficacia pratica dell'architettura sviluppata in scenari reali di movimento. I test eseguiti con l'algoritmo di localizzazione visiva ORB-SLAM3 hanno messo alla prova tutte le telecamere ambientali del visore, dimostrando la capacità del bridge di trasmettere un flusso continuo, coerente e privo di interruzioni verso ROS 2. In tutte le prove effettuate, il sistema ha garantito una perfetta sincronizzazione temporale e l'assenza di ritardi percepibili o perdite di frame. I dati visivi trasmessi hanno consentito di ricostruire i percorsi compiuti con notevole fedeltà rispetto al tracciamento nativo del visore:

- Percorsi a medio raggio: Sia con l'ottica centrale che con quella laterale destra, la traiettoria stimata ha ricalcato da vicino i movimenti reali, mantenendo scostamenti medi di pochi centimetri e seguendo con precisione i cambi di direzione e le curve.
- Orientamento della testa: I movimenti di rotazione e inclinazione dello sguardo dell'operatore sono stati riprodotti con estrema accuratezza lungo l'intera durata dei test, confermando l'assenza di sfalsamenti temporali nella ricezione dei fotogrammi.
- Percorsi estesi e rientro al punto di partenza: Nell'acquisizione a circuito chiuso con la telecamera sinistra, il sistema ha preservato la forma complessiva del percorso; l'aumento progressivo dell'errore registrato verso la fine della camminata riflette la naturale difficoltà di stimare distanze assolute usando una singola telecamera su tratti molto lunghi, senza però causare blocchi o perdite del tracciamento.

L'attività sperimentale conferma quindi che l'infrastruttura realizzata è solida, reattiva e pronta per essere integrata in applicazioni più complesse, fornendo una base affidabile per la condivisione dello spazio e l'interazione in tempo reale tra operatore e robot.

Capitolo 6

Conclusione e sviluppi futuri

6.1 Conclusioni

Negli scenari industriali moderni, la collaborazione umano-robot (HRC) e l'Industria 4.0 richiedono una condivisione continua e intuitiva delle informazioni nello spazio di lavoro. La Realtà Aumentata rappresenta lo strumento ideale per creare questo collegamento, ma l'assenza di una comunicazione diretta e nativa tra i visori AR e i sistemi robotici basati su ROS 2 crea un limite pratico notevole. Lo scopo principale di questo lavoro di tesi è stato proprio quello di superare questo ostacolo, sviluppando un "ponte" software flessibile ed efficiente in grado di mettere in comunicazione il visore Magic Leap 2 con l'ecosistema ROS 2. Sfruttando l'ambiente di sviluppo Unity e il pacchetto ROS-TCP-Connector, è stata realizzata un'applicazione a bordo del visore capace di acquisire ed esportare i flussi di tutti i principali sensori integrati. L'infrastruttura permette di trasmettere le tre telecamere di tracciamento ambientale, la telecamera frontale RGB ad alta risoluzione utilizzabile sia per lo stream video tradizionale sia per visualizzare la realtà mista con gli ologrammi sovrapposti, il sensore di profondità (Depth Camera) configurabile per operare a corto raggio (ad esempio per l'interazione fine delle mani) o a lungo raggio per la mappatura dell'ambiente, le quattro telecamere dedicate all'Eye Tracking per il monitoraggio dello sguardo dell'operatore, le unità inerziali (IMU) e il sensore di luce ambientale. Un elemento chiave del progetto è la gestione dinamica dei flussi: attraverso un semplice file di configurazione (config.yaml) gestito da un nodo su ROS 2, l'utente può decidere all'avvio quali sensori attivare, a quale risoluzione e con quale framerate. Questo evita di saturare la banda della rete locale (soprattutto in Wi-Fi), scongela problemi di surriscaldamento del visore e permette di cambiare setup senza dover modificare o ricompilare il codice ogni volta. La solidità del sistema è stata verificata sperimentalmente alimentando in tempo reale l'algoritmo ORB-SLAM3 in modalità monoculare con i dati delle World Cameras trasmessi tramite il bridge. Il confronto con il tracciamento del visore ha confermato la qualità dei dati trasmes-

si poiché le traiettorie stimate hanno seguito fedelmente i percorsi effettuati, registrando errori medi di traslazione nell'ordine di pochi centimetri (tra 4 e 9 cm su percorsi a medio raggio). Tale scostamento è dovuto principalmente al drift intrinseco dello SLAM monoculare; al contrario, il visore integra più telecamere, sensori IMU e un sensore di profondità, garantendo una precisione nettamente superiore. Inoltre i movimenti e le rotazioni della testa (Roll, Pitch, Yaw) sono stati riprodotti con accuratezza e senza ritardi percepibili, e anche su percorsi più lunghi a circuito chiuso (oltre 12 metri percorsi in circa 23 secondi), il flusso video non ha subito interruzioni o blocchi. In conclusione, il framework realizzato si è dimostrato un'infrastruttura solida, reattiva e modulare. Il rilascio dell'intero progetto come repository open-source su GitHub [3] permette di mettere questo strumento a disposizione della comunità scientifica e, in particolare, del laboratorio IAS-Lab dell'Università di Padova fornendo una base affidabile e pronta all'uso per sviluppare applicazioni avanzate in cui il visore non è un semplice schermo, ma un vero e proprio sistema sensoriale a supporto dell'interazione umano-robot.

6.2 Sviluppi futuri

L'architettura software modulare e i risultati sperimentali ottenuti in questo lavoro di tesi pongono le basi per estensioni ed applicazioni pratiche nell'ambito della robotica e della Collaborazione Umano-Robot (HRC). I principali sviluppi futuri si sviluppano lungo diversi fronti, tra cui tre particolarmente rilevanti. Il primo consiste nell'interazione bidirezionale e la visualizzazione delle intenzioni del robot. L'integrazione del canale di ritorno da ROS 2 verso l'ambiente grafico Unity consente di sfruttare il display a realtà aumentata per proiettare ologrammi contestualizzati direttamente nel campo visivo dell'operatore. Nello specifico, il sistema potrà visualizzare in tempo reale le decisioni computazionali del robot, le traiettorie pianificate e i volumi di sicurezza occupati durante la manipolazione. Questa trasparenza informativa riduce sensibilmente il carico cognitivo dell'utente e previene situazioni di pericolo durante compiti di cooperazione a stretto contatto. Inoltre, la medesima infrastruttura visiva può essere impiegata per la guida procedurale nelle catene di montaggio industriali, proiettando istruzioni passo-passo ancorate direttamente sugli oggetti fisici durante le sequenze di assemblaggio assistito. Il secondo fronte su cui è possibile sviluppare è il controllo intuitivo tramite gesture e interfacce miste. Sfruttando i flussi video e di profondità forniti dal bridge, è possibile estendere il framework integrando pipeline di tracciamento avanzato delle mani (Hand Tracking) e il riconoscimento dei gesti naturali. L'operatore potrà così impartire comandi diretti al robot, selezionare target nello spazio 3D o teleoperare manipolatori collaborativi in modo rapido e intuitivo, superando i vincoli delle tradizionali pulsantiere industriali. Tale approccio si inserisce nella direzione di interfacce ibride avanzate per l'HRC, in cui la realtà aumentata e il feedback gestuale cooperano per garantire

controllo dinamico ed evitamento attivo degli ostacoli [6]. L'ultimo fronte di sviluppo è la percezione egocentrica per sistemi indossabili ed esoscheletri. In pratica è possibile impiegare un visore AR non solo come dispositivo di interfaccia utente, ma come modulo di visione egocentrica (*head-mounted*) aggiuntivo a supporto di sistemi robotici mobili ed esoscheletri per arti inferiori [8]. La combinazione della prospettiva visiva della testa con i sensori posti alla base dell'esoscheletro può permettere di ricostruire mappe 3D, consentendo il rilevamento di ostacoli lontani o dislivelli del terreno e abilitando strategie adattive di pianificazione del cammino (*Environment-Adaptive Gait Planning*) assistite da feedback olografico.

Capitolo 7

Ringraziamenti

Desidero innanzitutto ringraziare il mio relatore, il Prof. Stefano Ghidoni, per avermi reso partecipe di questo stimolante progetto di tesi e per la disponibilità dimostrata durante il percorso. Un ringraziamento altrettanto profondo va al mio correlatore Dott. Matteo Terreran per avermi seguito passo dopo passo con straordinaria dedizione, per i preziosi consigli tecnici e per la prontezza nel chiarire ogni dubbio, guidandomi con competenza verso il compimento di questo lavoro. Un doveroso e sentito ringraziamento va al mio compagno di corso Federico Compagno, con cui ho condiviso le sfide e i progressi nello sviluppo del progetto. Ringrazio infinitamente la mia famiglia: i miei genitori Laura e Loris e mia sorella Lisa, che hanno creduto in me fin dal primo istante, motivandomi a dare il massimo e sostenendomi con affetto e sacrificio in ogni tappa della mia formazione. Un ringraziamento speciale va ai miei zii Giorgio e Francesca, per la loro generosità e per il sostegno concreto fondamentale in questi anni universitari. Grazie anche a tutto il resto della mia famiglia, che mi ha accompagnato e incoraggiato con affetto costante fin dai miei primi passi sui banchi di scuola. Un grazie sincero ai compagni della “Sacra prima fila”, con cui ho condiviso le fatiche e i successi di questi tre anni; in particolare ringrazio Edoardo e Diego, sempre presenti per confrontarsi, aiutarsi e alleggerire i momenti più intensi. Grazie anche a tutti gli amici che, anche da lontano, non mi hanno mai fatto mancare il loro supporto e la loro spinta motivazionale. Ringrazio anche Francesco, molto noto per essere il miglior Arrow Designer del pianeta, sei un vero amico e senza di te le frecce in figura 3.3 non sarebbero così belle.

Un ringraziamento speciale va a Vittoria, vero pilastro di questo percorso. Grazie per la tua pazienza infinita, per la fiducia incrollabile e per essere stata il mio porto sicuro in ogni momento di dubbio. Questo traguardo appartiene un po’ anche a te.

Bibliografia

- [1] EVST. «I 10 principali marchi di robot collaborativi per quota di mercato nel 2026,» visitato il giorno 10 ago. 2026. indirizzo: <https://www.evsint.com/it/top-collaborative-robot-brands>.
- [2] «Sito IKEA,» visitato il giorno 10 ago. 2026. indirizzo: <https://www.ikea.com/it>.
- [3] A. Rubert. «MagicLeap2_ROS2_connection: Sensor streaming bridge between Magic Leap 2 and ROS 2,» visitato il giorno 21 ago. 2026. indirizzo: https://github.com/alruby/MagicLeap2_ROS2_connection.
- [4] C. Campos, R. Elvira, J. J. Gómez, J. M. M. Montiel e J. D. Tardós, «ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,» *IEEE Transactions on Robotics*, vol. 37, n. 6, pp. 1874–1890, 2021.
- [5] A. Hietanen, R. Pieters, M. Lanz, J. Latokartano e J.-K. Kämäräinen, «AR-based interaction for human-robot collaborative manufacturing,» *Robotics and Computer-Integrated Manufacturing*, vol. 63, p. 101 891, 2020.
- [6] X. Yan et al., «A Complementary Framework for Human-Robot Collaboration With a Mixed AR-Haptic Interface,» *IEEE Transactions on Control Systems Technology*, vol. 32, n. 1, pp. 112–128, 2024.
- [7] E. Trombin, S. Tortora, E. Menegatti e L. Tonin, «Environment-Adaptive Gait Planning for Obstacle Avoidance in Lower-Limb Robotic Exoskeletons,» in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2024, pp. 13 640–13 647.
- [8] M. Mihailovic et al., «Egocentric Vision Module for Adaptive Gait Planning in Lower-Limb Exoskeletons,» in *Proceedings of the I-RIM Conference*, 2025, pp. 151–152.
- [9] D. Ungureanu et al., «HoloLens 2 Research Mode as a Tool for Computer Vision Research,» *arXiv preprint arXiv:2008.11239*, 2020.
- [10] J. C. Dibene e E. Dunn, «HoloLens 2 Sensor Streaming,» *arXiv preprint arXiv:2211.02648*, 2022.

- [11] M. Leap. «Magic Leap 2,» visitato il giorno 10 ago. 2026. indirizzo: <https://www.magicleap.care/hc/en-us/articles/4416076504589-Controller-LED-Guide>.
- [12] M. Leap. «Magic Leap Waveguide engineering,» visitato il giorno 10 ago. 2026. indirizzo: <https://www.magicleap.com/waveguide-engineering>.
- [13] M. Leap. «Magic Leap Waveguide,» visitato il giorno 10 ago. 2026. indirizzo: <https://www.magicleap.com/waveguides>.
- [14] M. Leap. «Magic Leap Developer Docs,» visitato il giorno 21 ago. 2026. indirizzo: <https://developer-docs.magicleap.cloud/docs/guides/ml2-overview>.
- [15] Unity Technologies. «ROS-TCP-Connector: Unity Integration for ROS,» visitato il giorno 28 ago. 2026. indirizzo: <https://github.com/unity-technologies/ros-tcp-connector>.
- [16] M. Leap. «Magic Leap Getting Started,» visitato il giorno 21 ago. 2026. indirizzo: <https://developer-docs.magicleap.cloud/docs/guides/unity-openxr/getting-started/openxr-unity-getting-started/>.
- [17] M. Grupp. «evo: Python package for the evaluation of odometry and SLAM,» visitato il giorno 4 set. 2026. indirizzo: <https://michaelgrupp.github.io/evo/>.
- [18] S. Umeyama, «Least-squares estimation of transformation parameters between two point patterns,» *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, n. 4, pp. 376–380, 1991. doi: 10.1109/34.88573.